

Declaration of originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 2397D, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: *May 16, 2025*

Proforma

Candidate Number: **2397D**
Title: **IsabelleGym: A Learning Environment for Reinforcement Learning With Isabelle/HOL**
Examination: **Computer Science Tripos – Part II, 2025**
Word Count: **11995**
Code Line Count: **3450**
Project Originator: **Dr Wenda Li**
Project Supervisors: **Dr Wenda Li, Prof. Lawrence Paulson**

Original aims of the project

Reinforcement learning (RL), coupled with interactive theorem provers (ITPs) has shown promise in automating formal theorem proving. Isabelle/HOL, a mature and popular ITP, has notably lagged behind its counterparts in RL-based automation, largely due to the lack of a robust and efficient learning environment abstracting away the complexities of ITP interaction. This project aimed to fill this void by developing a “gym” environment for training RL agents to automate proofs in Isabelle. In this pursuit, it would build infrastructure for external interaction with Isabelle and demonstrate the feasibility of said infrastructure via RL tasks conducted within it.

Work completed

The project was greatly successful, culminating in the development of IsabelleGym – a learning environment incorporating many of the state-of-the-art features of environments for other ITPs. A novel external interaction pipeline was architected for Isabelle, substantially enhancing usability over existing systems and delivering impressive performance. IsabelleGym significantly advances the RL landscape for Isabelle, bearing advanced features such as vectorised environments. The core success criteria were met and greatly expanded upon. Notably, the N -puzzle was formalised in Isabelle/HOL, with an agent coupling Monte Carlo Tree Search and temporal difference learning trained to solve the puzzle in IsabelleGym.

Special difficulties

None

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Prior related work	2
1.3	Aims	2
2	Preparation	4
2.1	Isabelle	4
2.1.1	Isabelle/Pure and Isabelle/HOL	4
2.1.2	Proof styles: Tactics vs Isar	4
2.1.3	The Prover IDE (PIDE) architecture	5
2.1.4	Bridging the gap with external ATPs: Sledgehammer	6
2.2	Formalising ITP as a Markov Decision Process (MDP)	6
2.2.1	MDP components	6
2.2.1.1	Proof state space	6
2.2.1.2	Action space	7
2.2.1.3	Transition function	8
2.2.1.4	Reward function	8
2.2.1.5	Discount factor	8
2.2.2	Role of the agent	8
2.2.3	Role of the environment	8
2.3	Survey of existing ITP-ML environments	9
2.3.1	Coq	9
2.3.2	Lean	10
2.3.3	Isabelle	10
2.4	Requirements analysis	11
2.4.1	Methodology	12
2.4.2	Programming languages	12
2.4.3	Code management and testing tools	12
2.5	Starting point	13
3	Implementation	14
3.1	Novel interaction pipeline: REPL interface	14
3.1.1	REPL Commands	14
3.1.2	PIDE interaction	15
3.1.3	Inter-process communication (IPC)	18
3.2	Gym environment	18
3.2.1	API design	19
3.2.2	Environment state saving and restoring	19
3.2.3	Proof state extraction	21
3.2.4	Sledgehammer integration	21
3.3	Training an RL agent to prove theorems within Isabelle/HOL	22

3.3.1	Problem formalisation – N -puzzle	22
3.3.2	Combining AlphaZero-style Monte Carlo Tree Search with temporal difference learning	23
3.3.2.1	Temporal difference (TD) learning	24
3.3.2.2	MCTS	25
3.3.3	Agent training and evaluation	26
3.4	Vectorised environments	27
3.4.1	Gym API changes	27
3.4.2	Possible approaches	28
3.4.3	Implementation	29
3.4.4	Example usage	29
3.5	Multi-agent RL support	30
3.5.1	Multi-agent RL and theorem proving: collaborative vs competitive . . .	30
3.5.2	Agent-Environment Cycle (AEC) support	31
3.5.3	Parallel agents	32
3.6	Repository overview	32
4	Evaluation	33
4.1	Usability	33
4.1.1	Comparison with existing learning environments for Isabelle/HOL	33
4.1.2	Advancements in the Isabelle-RL environment landscape during the development of IsabelleGym	34
4.1.3	Comparison with existing learning environments for other ITPs	35
4.2	Correctness	35
4.2.1	Unit and integration tests	36
4.2.2	AFP simulations	36
4.3	Performance	37
4.3.1	Comparing IsabelleGym interaction with raw Isabelle	37
4.4	Work completed	38
5	Conclusions	39
5.1	Lessons learnt	39
5.2	Future work	39
	Bibliography	41
	A Isabelle/ML, Isabelle/Scala and scala-isabelle	44
	B N-puzzle formalisation in Isabelle/HOL	45
	C Demonstration of collaborative multi-agent theorem proving	47

1 Introduction

Interactive theorem provers (ITPs)¹ are powerful software systems that collaborate with users to construct formal proofs [7]. Recent advances [1, 5, 39] have shown that it is possible to replace the human users with agents that train via reinforcement learning (RL) to interact with ITPs and automate proof construction. Despite its maturity and popularity, very little work in this area has been applied to the Isabelle/HOL ITP. While efficient environments for applying RL algorithms have been constructed for other ITPs (e.g. [lean-gym](#) [21], [CoqGym](#) [41]), a standardised learning environment for Isabelle/HOL is yet to emerge. This raises the following question: is it possible to create a learning environment for Isabelle/HOL with comparable efficiency, standardisation and ease of use?

1.1 Motivation

Automated theorem proving (ATP)², whereby a computer program produces a formal proof for an assertion without any human intervention, has long been a goal since the advent of machine learning (ML) [18]. However, for many of the software and mathematical systems that we are interested in modelling the sizes of the (often infinite) search spaces and complexity of proof states traversed during complex proofs has meant intense embedding of domain-specific knowledge and careful optimisation has been needed to make ATPs remotely feasible. This chasm has allowed the development of ITPs [7], distinguished from ATPs via their incorporation of user-computer interaction during proof generation. Over time, much of the work in the field of ATPs has migrated to automating parts of proofs in ITPs with external tools such as hammers being commonplace in the mature ITPs. Nonetheless, the need for human intervention and domain-specific intuition when using ITPs remains a significant bottleneck during proof generation.

RL offers an alternative approach wherein a general algorithm can be used to learn how to prove theorems in specific domains. An agent can be trained to interact with an ITP in order to generate a proof script. This approach has proved to be successful with TacticZero [39] showing that RL with the HOL4 [10] ITP can be used to generate proofs without requiring examples and AlphaProof [1] demonstrated that pairing a pre-trained large language model with the AlphaZero [27] RL algorithm can generate proofs in Lean 4 [17], achieving silver-medal performance on the 2024 International Math Olympiad problems. The main requirement for an RL approach is a “gym” environment that enables efficient communication between the agent and the ITP, abstracting away the complexity of the ITP while exposing the internal proof states and available actions (which vary immensely between ITPs). Isabelle/HOL, with its rich feature set and large proof library (the [Archive of Formal Proofs \(AFP\)](#) [8]), is a ripe candidate for RL-based proof generation. However, the lack of a standardised learning

¹In this dissertation, ITP refers to interactive theorem prover/proving. Whenever used, the correct expansion will be obvious from the context.

²The same abbreviation convention as for ITP is used.

environment targetting it has hindered progress in this area.

1.2 Prior related work

A more detailed survey of existing work is provided in Section 2.3 but a brief overview is provided here.

Multiple established ITPs now co-exist, each with their own strengths and underlying logical foundations. Coq is widely used for program verification and has [CoqGym](#) [41], providing a large corpora of proofs and a learning environment, and Tactician [2], a tool for tactic prediction that also provides an ML environment. Lean is a more recent ITP that has rapidly gained popularity particularly in formalising mathematics and has [lean-gym](#) [21], providing a read-eval-print-loop (REPL) interface for interacting with Lean, and [LeanDojo](#) [42], providing a more structured gym environment.

For [Isabelle/HOL](#), there is [PISA](#) [13], which provides an environment for agents to interact with [Isabelle/HOL](#) incrementally. Despite there being an official [Isabelle/Scala](#) pipeline designed for external interaction with Isabelle by prover integrated development environments (PIDEs), [PISA](#) is based on [scala-isabelle](#), an unofficial pipeline for interacting with Isabelle via a Scala process. Highlighting the recognition of the ITP community that a novel pipeline for interaction with Isabelle is needed, a project (Isabelle/RL) was proposed at the latest AITP conference [40] to create RL tooling for Isabelle. As of the start of the IsabelleGym project, Isabelle/RL has only completed the first stage of extracting training data from Isabelle.

1.3 Aims

Hence, the aim of this project is to create IsabelleGym, a standardised learning environment utilising the official [Isabelle/Scala](#) pipeline for interaction with [Isabelle/HOL](#). The high-level objectives amidst the creation of IsabelleGym are as follows:

Core Aims

- **Novel interaction pipeline:** Architect an external interaction mechanism for [Isabelle/HOL](#) using [Isabelle/Scala](#) rather than [scala-isabelle](#), providing a REPL front-end.
- **IsabelleGym:** Construct an RL gym providing an abstracted interface for RL agents to construct proofs³ with [Isabelle/HOL](#) via the aforementioned pipeline.
- **Functionality demonstration:** Train an agent in IsabelleGym via RL to prove unseen theorems in [Isabelle/HOL](#).

³In the proposal, support for forward and declarative proofs were divided between the core and extensions. However, Isar (see Section 2.1.2) and our REPL implementation were flexible enough to support both with minimal changes so both were tackled together.

Extension Aims

- **Vectorised environments:** Manage multiple independent environments, handling collections of actions in parallel.
- **Multi-agent RL support:** Enable multiple agents to interact with a single environment and each other such that they can collaborate or compete to prove theorems.

2 Preparation

IsabelleGym combines the fields of ITP and RL, with each bearing significant theoretical complexities and nuances that influence the design choices within the project. In this chapter, we:

- Provide the necessary background concerning [Isabelle/HOL](#) (Section 2.1).
- Formalise the considerations behind applying RL to ITP, specifically abstracting ITP as a sequential decision making problem (Section 2.2).
- Explore the landscape of existing ITP-ML environments (Section 2.3).
- Conclude by refining the requirements and selected tools using the prepared knowledge (Section 2.4) and detailing the project starting point (Section 2.5).

2.1 Isabelle

2.1.1 Isabelle/Pure and Isabelle/HOL

Isabelle itself is a framework for constructing deductive systems, providing a meta-logic **Pure**. The meta-logic serves as the formal infrastructure for defining concrete object logics, including tools for specifying syntax, semantics and inference rules. For example, it provides meta-operations such as meta-implication ($\Rightarrow$) [20]. Upon this meta-logic, an object logic is implemented defining syntax, axioms and inference rules within the meta-logic. [Isabelle/HOL](#) is the most ubiquitous object logic within Isabelle, implementing classical higher-order logic (HOL), and thus is the focus of this project.

2.1.2 Proof styles: Tactics vs Isar

Early proofs in Isabelle used tactics, functions in Isabelle’s kernel language ML that directly manipulate the proof state. This procedural style, chaining tactic calls to imperatively transform the proof terms, meant proof scripts consisted of code snippets providing operational instructions for the prover, allowing fine-grained control but resulting in complex scripts difficult for humans to read.

Intelligible semi-automated reasoning (Isar) [38] addressed the limitations of the tactic-based approach by providing a structure proof language that allowed writing proofs in a declarative style, starting from the goals and deconstructing into subgoals, that closely resembled the structure of traditional mathematical proofs. This was achieved by separating the proof document from the proof state, with Isar commands manipulating the proof document and the underlying Isar VM managing the manipulations to the proof state. Keywords such as **proof**, **qed**, **assume**, **show** (used when stating intermediate goals) lay out the structure of the proof and individual proof steps are typically concluded using **by** <method> commands which invoke either basic reasoning steps or powerful automated tactics such as **simp** (simplification), **blast** (tableau

prover) or Sledgehammer (see Section 2.1.4). As a result of reconciling human readability with machine checkability, Isar has become the target of virtually all modern proof development in Isabelle/HOL.

Isar still allows embedding tactic scripts via the `apply` command but they are considered an improper sublanguage and are discouraged for production proofs.

2.1.3 The Prover IDE (PIDE) architecture

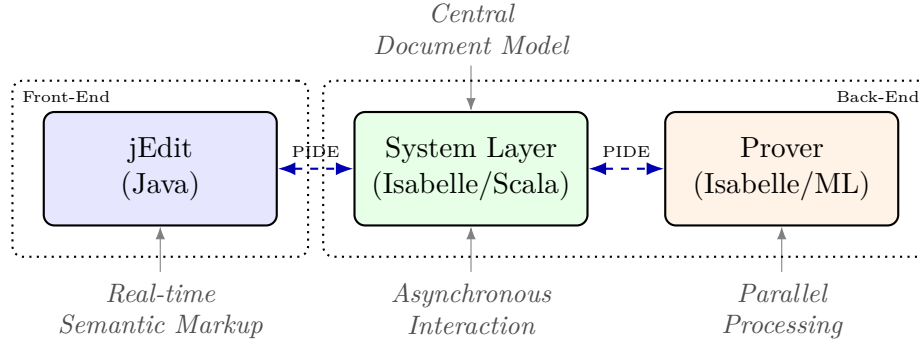


Figure 2.1: Architecture of Isabelle/PIDE

The interactivity of the Isabelle system is facilitated by PIDE [35, 37], a framework offering a **document-oriented** interaction model. Early ITPs presented a traditional command-line interface processing a stream of commands sequentially. In contrast, PIDE presents a formal proof document maintained by the prover as the fundamental target of interaction. The document itself is a directed acyclic graph (DAG) of theory nodes, with connections representing the logical dependencies between them. Under this model, AFP can be represented as just one large mathematical document increasing the ease of user interaction with this huge proof corpora. This model enables two key advancements:

- **Pervasive parallelism:** The structured view of the proof document allows identifying independent subgoals which can then be checked by the prover concurrently. Thus, PIDE can leverage multi-core processors much more effectively than the previous sequential models. It is described as “pervasive” as this parallelism is built-in to the processing model and not a later optimisation.
- **Asynchronicity:** PIDE separates the user document from the prover document, allowing non-blocking interaction wherein the user can continue to make edits while the prover is still processing earlier updates. When text edits are made to the user document, these are converted into structured updates issued to the prover via PIDE. The prover uses these updates to amend its internal model of the proof document and asynchronously sends back formal markup reports (containing semantic highlighting, errors, processing status) back to the editor.

The standard front-end for PIDE is Isabelle/jEdit [36]. It leverages information obtained from the PIDE backend to offer semantic editing (awareness within the editor of the structure of theories and the statuses of different parts of the proof document). This editing experience includes

real-time markup whereby the proof documents are continuously annotated with information such as errors and the results of proof steps.

2.1.4 Bridging the gap with external ATPs: Sledgehammer

As discussed in Section 1.1, the gap between ATPs and ITPs has decreased over time by reframing ATPs as powerful tactics that can be invoked from within a proof in an ITP. Isabelle testifies to this by providing Sledgehammer [3, 19], a tool that can automatically prove or refute the current goal by invoking external ATPs (e.g. E [26], Vampire [31]) and satisfiability-modulo theories (SMT) solvers. Upon applying the `sledgehammer` command within a proof, **relevance filtering**, analysing the current proof context (see Section 2.2.1.1) to heuristically select a subset of relevant facts (lemmas, theorems, definitions or axioms). Various filters are available with the traditional Meng-Paulson filter assigning scores to every available fact based on the number of constants it shares with the goal. The problem and the selected relevant facts are translated into first-order logic¹ by the internal proof engine (aiding proof reconstruction in Isabelle). The ATPs are then run concurrently on the translated formalism. If any find a proof within a set time limit, Isabelle attempts to reconstruct the proof within its own trusted engine – a necessary step to ensure soundness amidst untrusted external systems and the possibility of unsound translations.

For RL systems, Sledgehammer is relevant twofold. Firstly, it can be regarded as a powerful, albeit time-consuming action, that the agent can execute to prove a goal. In addition, Sledgehammer’s internal relevance filtering step can be used as a source of heuristic information that the agent can leverage to improve its own premise and tactic prediction capabilities.

2.2 Formalising ITP as a Markov Decision Process (MDP)

The application of RL to ITPs requires the formalisation of an ITP as a sequential decision-making problem where the ITP is responsible for managing sequential state and the agent is responsible for ingesting the state and outputting actions to manipulate this state. A mathematical formalism enabling this representation is the Markov Decision Process (MDP) [33] which models executing actions (i.e. decisions) sequentially in a domain where the outcomes of these actions have some known probability distribution. It is defined as a tuple (S, A, P, R, γ) .

2.2.1 MDP components

2.2.1.1 Proof state space

For ITP, a state $s \in S$ represents a proof state, i.e. a snapshot of the active proof goals (logical formulae that need to be proven) and the current proof context. ITP forms a complex symbolic domain (characterised by human-readable formal symbols and pre-defined logical rules as opposed to neural domains often containing black-box manipulations of numerical embeddings). As such, the representations used when applying ML to ITP tend to differ significantly from conventional applications.

¹Most modern ATPs require the input to be in FOL whereas most ITPs support logics richer than FOL. Isabelle/HOL supports polymorphic HOL and so in practice, Sledgehammer enjoys limited success when non-trivial higher-order steps are required.

In the context of Isabelle/HOL specifically, the proof state has formal grounding within Pure. If the currently open subgoals are $\phi_1, \phi_2, \dots$ and the main goal is ψ , the proof state can be represented via a sequent within the meta-logic,

$$\Phi \vdash \llbracket \Phi \rrbracket \implies \phi_1; \phi_2; \dots$$

where Φ represents the meta-level assumptions (i.e. the proof context).

Each subgoal can then be represented as a sequent in the object logic HOL,

$$n. \llbracket A_1; A_2; A_3 \rrbracket \rightarrow G$$

where n is the index of the subgoal, A_1, A_2, A_3 are local assumptions (HOL formulas) and G is the conclusion (HOL formula) to be proven under these assumptions.

Schematic variables can also be found in the proof state (e.g. `?x`) and act as placeholders for unknown proof portions and are instantiated via higher-order unification. The proof context forms the stable background environment, including:

- Fixed local variables (`fix`).
- Local assumptions/facts (`assume`, `note`, `have`)
- Imported theorems, axioms, constants, etc. from libraries (e.g. `Main`)

Each intermediate step in the interactive proof then corresponds to a valid statement about the proof state, constructing a meta-level derivation tree. The list of subgoals can be viewed as the frontier of this derivation tree, with backward proofs starting from the root and gradually decomposing the leaf nodes into smaller subgoals. This culminates in the completed proof, which is a meta-level theorem where the list of open subgoals is empty, directly mirroring natural deduction systems.

Transforming this symbolic proof state into a numerical representation for deep learning poses various challenges due to the need to capture both the syntactic structure and the semantic connections between the components (e.g. dependencies between hypotheses and goals). Several approaches have been employed including linearising and using sequence-based models, operating over the abstract syntax tree of the proof state with tree-structured models (e.g. TreeLSTM) and utilising graph neural networks.

2.2.1.2 Action space

An action $a \in A$ corresponds to proof step that the agent can take to manipulate the proof state. For Isabelle, this is either applying a procedural tactic or declarative Isar command. Actions often have arguments (e.g. specific tactic names, references to theorems), critically allowing references to elements from libraries and so the action space is combinatorial and can even expand during the proof as new definitions are introduced. Success in theorem proving is contingent on premise selection: the ability to effectively filter and apply existing knowledge. Initial work in this area used heuristics measuring syntactic similarity (e.g. Sledgehammer) but modern approaches have shifted to machine learning approaches (e.g. NeuroTactic, MagnusHammer). Flat classification (where each combination of command and argument is treated as a separate

action) quickly becomes infeasible due to combinatorial explosion. Hence, a clear trend of tight integration between tactic prediction and premise selection has emerged with the ML systems using a hierarchical approach where either relevant commands or premises are selected first and then the other is selected conditioned on the first choice.

2.2.1.3 Transition function

$P_a : S \times S \rightarrow [0, 1]$ defines the probability of transitioning from state s to state s' for a given action a . In ITP, this often reduces to a deterministic function where applying a given action to a given state always results in the same unique state. Non-determinism could arise from the use of external non-deterministic tools.

2.2.1.4 Reward function

$R : S \times A \times S \rightarrow \mathbb{R}$ defines the reward received by the agent for reaching state s' from state s by taking action a . In ITP, designing an effective reward function is particularly challenging as the most intuitive reward signal (i.e. a large positive reward received upon completion of a proof) leads to an extremely delayed and sparse reward signal that gives no indication about which intermediate actions to select. **Reward shaping** is a technique often used with ITPs [39], wherein small intermediate rewards are given for actions that generally lead closer to a complete proof, e.g. reducing the number of open subgoals.

2.2.1.5 Discount factor

$\gamma \in [0, 1]$ determines the balance in the importance of immediate and future rewards (γ^n is multiplied to the reward for timestep n when calculating discounted measures of the reward so a γ closer to 1 gives more weight to future rewards). A γ value of 1 is possible for episodic MDPs where termination is guaranteed after some bounded number of steps. However, in continuous or possible infinite MDPs (such as ITPs where useless steps can be used to extend proofs indefinitely), γ is bounded below 1 to ensure the convergence of the discounted reward sum. With the ultimate goal of completing a proof, intuition suggests the discount factor should be close to 1 (avoiding neglecting future rewards). However, combining long proof horizons with high discount factors can exacerbate the problem of sparse rewards.

2.2.2 Role of the agent

Given an MDP, the RL agent can specify a policy function $\pi : S \times A \rightarrow [0, 1]$ with $\pi(s, a)$ specifying the probability that the agent will take action a in state s . The behaviour of the agent is completely specified by the policy and selecting this collapses the MDP to a Markov Chain. The goal for the agent is to find the π that maximises the expected value of some combination (e.g. expected discounted sum) of the probabilistic rewards.

2.2.3 Role of the environment

The presence of such an abstraction enables environments for many different RL problems to provide a common interface. This allows building a “gym environment” which implements this common interface for a specific RL problem. It is this concept that has led to projects such as [Gymnasium](#) [32] (formerly [OpenAI Gym](#) [4]), which combines gym environments for numerous problems that all implement the same interface into one package. Users can then

test and benchmark their RL algorithms on a wide variety of problem domains without having to change the core RL algorithm. Such gym environments typically [4] provide the following functions:

- Initialise the environment and provide the initial state as an observation to the agent.
- Allow the agent to select an action to perform in the current state.
- Apply the action chosen by an agent in the current state and return the new state, reward², whether the new state is terminal and any additional information associated with the application of the last action.
- Handle environment termination.

2.3 Survey of existing ITP-ML environments

The main ITPs and the subjects of the majority of ITP-ML research have been Coq, Lean and Isabelle, so we explore the architectures of the existing environments for these ITPs.

2.3.1 Coq

RL efforts for Coq were spearheaded by [CoqGym](#) [41] focused on providing a huge dataset of proofs that could then be used to train supervised learning models. Specifically, [CoqGym](#) processed a large number of open source Coq proof libraries, extracting the various intermediate proof states and corresponding tactics selected by human experts. Initial systems utilising this dataset solely focused on tactic prediction (e.g. ASTactic’s TreeLSTM) but the field since shifted to predicting premises also (e.g. Proverbot9001) and more recently, incorporate large language models (e.g. PALM combining LLMs with symbolic repair).

Tactician [2] is a tool capable of tactic prediction and completing proofs in Coq. Tactician works by capturing the tactics utilised by the user during proof synthesis, recording the tactic itself and the proof states before and after the execution of the tactics. It also provides a machine learning platform enabling external agents to interact with Coq via the Tactician API [29]. An additional Python wrapper, PyTactician [23], is provided for ease of interacting with this API, facilitating the following features:

- **Communication abstraction:** Low-level details of the Cap’n Proto serialisation protocol and the IPC mechanisms for communication with Coq are hidden from the user, allowing interaction with Python objects and methods rather than the low-level protocol data.
- **Dataset interaction:** Tools are provided for interacting with the datasets collected by Tactician, enabling easy supervised training of new models.
- **Agent interaction:** Data structures representing proof elements and functions for sending commands and receiving the corresponding proof state changes are provided, democratising access to data collection and communication with Coq.

²The reward function is defined within the environment itself but these environments can be extended to provide custom reward functions.

2.3.2 Lean

Initial machine learning environments for Lean such as [lean-gym](#) focused on providing a simple read-eval-print-loop (REPL) interface for simple experimentation. However, this simple interaction method was superseded by LeanDojo [42] which offers a structured mode of interaction:

- `inititalize(theorem)`: Returns the state of the environment after initialising it to prove a specific theorem, with the state encoded as a string containing the current proof goals and context. `test`
- `run_tac(state, tactic)`: Run a tactic on a given state and return the new state. Error handling is performed with an error state being returned if the input state was an error state or the tactic application failed.

LeanDojo is more robust than [lean-gym](#), reducing the number of ground truth proofs misjudged as erroneous from 21.1% to 1.4%. It provides the LeanDojo Benchmark containing almost 100,000 proofs extracted from `mathlib`.

2.3.3 Isabelle

In comparison to the environments available in Coq and Lean, the Isabelle ecosystem is notably less mature. [PISA](#) demonstrated the feasibility of constructing an RL environment for Isabelle, producing a proof corpus similar to those available in CoqGym and LeanDojo by mining from the Isabelle standard library and [AFP](#). However, its reliance on [scala-isabelle](#) for interacting with Isabelle adds significant overhead due to duplicating the internal structures of Isabelle. In addition, not using the PIDE protocol limits the concurrency that can be exploited by the environment, particularly in terms of the parallel processing of independent subgoals within the same theory. The fundamental assumption underpinning the MDP formalism (see Section 2.2) is the Markov property: any future transformation of the state given the current state and action is independent of the previous sequence of states and actions. The proof states extracted via [PISA](#) are the simple user-facing states that do not include the full proof context. This simplification means that RL agents must condition on the previous proof steps in order to determine the true state of the proof, significantly corrupting the boundary of responsibility between the agent and the environment.



Figure 2.2: The intermediate result established by the `have` command (`1+1 = (2::int)`) in the Isar proof snippet (left) is not visible in the surface state extracted by PISA (right), despite being part of the full proof context available internally in Isabelle/ML.

[MagnusHammer](#) [16] represents the state-of-the-art in learned premise selection for Isabelle surpassing [Sledgehammer](#) for the single-step task and achieving best-in-class performance on the multi-step task when combined with [Thor](#) [12] both on the [PISA](#) benchmark. Training

necessitated the extraction of a huge dataset of proof state-action pairs using PISA. The relative sparsity in machine learning efforts for Isabelle can be partially attributed to the lack of a robust environment for interaction with Isabelle making development and experimentation difficult. Hence, there is a clear gap that IsabelleGym can fill.

2.4 Requirements analysis

Recall from the Proposal and Section 1.3 that the project consists of three core deliverables:

- **REPL interface** based on a novel Isabelle/Scala pipeline for interaction with Isabelle/HOL.
- **Gym environment** enabling RL training of agents to construct proofs in Isabelle/HOL via new pipeline.
- **Theorem-proving RL agent** trained via IsabelleGym, demonstrating the environment’s functionality.

From the preparation, we can further refine the requirements.

Refined Requirements

Study of Isabelle (Section 2.1):

- Utilisation of the PIDE architecture is essential for full exploitation of Isabelle’s parallelism and so the use of Isabelle/Scala is further justified. This parallelism and asynchronicity is at odds with the single-step models of traditional gym environments, so IsabelleGym must find a way to leverage maximum parallelism while providing a simple interface with robust error reporting.
- IsabelleGym must allow invoking Sledgehammer as an action, allowing the agent to leverage external ATPs and creating an increased focus on the complex parts of the proof.

Reframing ITP as an MDP (Section 2.2):

- Full knowledge of the proof state, including both the open subgoals and the proof context, is essential to maintain the Markov property. However, the full context is often not necessary for the agent to select the next action. Hence, IsabelleGym must allow the agent to specify the level of detail required in the proof state.

Exploration of existing environments (Section 2.3):

- There must be a concept of state that can be saved and restored, allowing the agent to quickly explore the proof space and then return to an earlier checkpoint.
- All of the major ITP-RL environments support mining existing proofs for training data. IsabelleGym must allow extracting proof-state action pairs from a given proof.

2.4.1 Methodology

Given that each of the core deliverables for this project build upon the previous deliverable, the **spiral model** was adopted. The order of the first three core phases were forced. Subsequently, priority was given to extensions based on the trade-off between the complexity of the task and the expected benefits for usability and performance. The frequency of similar implementations in the literature served as a useful indicator of expected risk.

2.4.2 Programming languages

Scala served as the back-end language for communication with Isabelle, allowing the use of Isabelle/Scala and PIDE. As hybrid functional and object-oriented language, it allowed leveraging functional patterns such as composition while still accommodating the convenience of object-oriented approaches for managing state. Low-level interaction functions were written in Standard ML which could then be embedded within proofs.

Python was selected as the front-end language for exposing the actual REPL interface, gym environment and implementing RL algorithms. The choice of Python was motivated by its overwhelming popularity in the machine learning community, with the majority of ML libraries and frameworks offering Python APIs. In addition, Python’s dynamic typing and interpreted nature greatly aid rapid prototyping and experimentation.

2.4.3 Code management and testing tools

Testing was managed on the Python side using **pytest**. In order to reduce the burden of writing and maintaining tests, the expect test pattern [28] was used via the Python **expecttest** library. In this paradigm, we write tests that produce a specific output and then these outputs are compared against the expected outputs. Upon any changes to the outputs, the library automatically updates the expected outputs and we check that the new outputs are correct.

Git was used as the primary version control system, with the repository and dissertation writing actively backed up to GitHub. Both were also held in folder set up to automatically synchronise with OneDrive, ensuring continuous backup. For the dissertation, using Git allowed reviewing changes made before committing, preventing accidental deletions. Testing was done via pre-push hooks, allowing local testing (avoiding repeated expensive setup of Isabelle in a remote workflow) while avoiding having to run tests for every commit (as in a pre-commit). This ensured that code pushed to the repository was always in a working state.

Formatting was handled by a linter (**pylint**) and formatters (**black** and **isort**) on the Python side, the formatter **scalafmt** for Scala and **Prettier** for other files (e.g. YAML, Groovy). Autoformatters were configured in the IDE for this project to run upon saving and final formatting checks were conducted via a **super-linter** GitHub workflow (combines multiple linters including checking for leaks of secrets in pushed code) running on every push to the repository. Scala is statically typed, and **mypy** provided static type checking for Python, eliminating a whole class of bugs.

2.5 Starting point

Prior to beginning the project, I had some limited experience with SML and Scala but both were relatively easy to work with due to having worked with other functional and object-oriented languages (e.g. OCaml, Java) previously. Python was familiar to me and I had experience managing large projects with it. I had very little previous experience with [Isabelle/HOL](#) and so the ACS [Proof Assistants](#) module was used to provide the foundational knowledge required for the project.

The project builds on the Isabelle codebase [\[11\]](#) which is available to extend under a BSD-style license (imposes minimal restrictions aside from needing to preserve the copyright notice). No work on the project was conducted before Michaelmas 2024.

3 Implementation

3.1 Novel interaction pipeline: REPL interface

Requirements Recap

Isabelle/Scala and PIDE The communication pipeline should be built on the official Isabelle/Scala pipeline and utilise the PIDE architecture.

REPL front-end Interaction should be possible in REPL style, with the user able to send commands to Isabelle and receive immediate responses. The nature of REPL interaction inherently presents a synchronous interface, with feedback presented before the next command can be executed. Hence, the REPL will need a mechanism for managing the asynchronicity of the underlying PIDE architecture.

3.1.1 REPL Commands

The REPL is a thin front-end to demonstrate the functionality of the new Isabelle/Scala interaction pipeline. While mainly focused on passing Isar snippets to Isabelle, some additional meta-commands are provided to manage the interactive session.

REPL Command	Function
<code><isar snippet></code>	Add the Isar snippet to the current theory.
<code>enter <name></code>	Switch to editing theory <code><name></code> (creating if new).
<code>rollback</code>	Undo the last Isar snippet insertion.
<code>source</code>	Display the source Isar code of the current theory.
<code>help</code>	Describe available commands.
<code>exit</code>	Exit the REPL session.

Table 3.1: REPL commands.

Design Choice

Stack architecture This interface reduces the editing experience to adding or popping insertions. Alternative approaches such as allowing jumping back to specific insertion points to start inserting from the middle of the theory were considered but deemed to be an unnecessary complication for the REPL interface. We are not aiming to provide a full editor experience but rather an interface for agents to conjecture, characterised by exploring the proof space and backtracking upon hitting a dead end. Hence, this stack interface is more aligned with the needs of RL agents.

Foreshadowing: This choice allows an efficient implementation of state saving and restoring in the gym environment (see Section 3.2.2).

```

[no theory entered (use 'enter <theory>')] >>> enter Test

[theory = Test] >>> theory Test imports Main begin

[theory = Test] >>> theorem "1+2=3"
proof (prove)
goal (1 subgoal):
  1. 1 + 2 = 3

[theory = Test] >>> proof -
proof (state)
goal (1 subgoal):
  1. 1 + 2 = 3

[theory = Test] >>> show ?thesis
proof (prove)
goal (1 subgoal):
  1. 1 + 2 = 3

[theory = Test] >>> by auto
show 1 + 2 = 3
Successful attempt to solve goal by exported rule:
  1 + 2 = 3
proof (state)
this:
  1 + 2 = 3

goal:
No subgoals!

[theory = Test] >>> qed
theorem 1 + 2 = 3

```

Figure 3.1: Example interaction with the REPL interface, demonstrating entering a new theory before stating and proving a theorem within it interactively.

3.1.2 PIDE interaction

A session in Isabelle groups together multiple theories in a construct similar to a project in a programming language. Sessions can have hierarchical structure with the **Pure** session, corresponding to the meta-logic of Isabelle, being the root of all sessions. Utilising an Isabelle server (allows managing sessions), we begin a headless (without a front-end interface like **jEdit**) HOL session and the role of our pipeline now is to allow the user to update this session externally with their own theories.

The overall architecture is based on transforming the Isar snippets passed in by the user into structured `Document.Edit_Text` commands that can then be sent to the prover via the PIDE interface.

```

object Document {
  ...
  object Node {
    ...
    sealed abstract class Edit[A, B] { ... }
    // Raw text edits
    case class Edits[A, B](...) extends Edit[A, B]
    // Imports
    case class Deps[A, B](...) extends Edit[A, B]
    // Indicates whether theory is user-facing/required
    case class Perspective[A, B](...) extends Edit[A, B]
    ...
  }
  type Edit_Text = (Node.Name, Node.Edit[Text.Edit, Text.Perspective])
  ...
}

```

Figure 3.2: Simplified Isabelle/Scala `Document.Edit_Text` structure, showing the three types of edits.

As mentioned in Section 2.1.3, a document is a DAG of theory nodes. Hence, we maintain one node per theory and edits are applied to the corresponding theory node. Despite initially appearing straightforward, various subtleties arise from this use of PIDE.

Handling asynchronicity while leveraging parallelism The results of the commands created from the REPL edits are returned from the prover asynchronously. As the REPL (and future gym environment) requires a synchronous interface, outputs are blocked until all commands for the theory node have completed. This approach still allows the commands to execute in parallel (e.g. if a large REPL update was provided containing multiple independent subgoals) while maintaining robust error reporting (a non-blocking approach would allow error reports to be delayed until after other REPL inputs). Various delay configurations were tuned to make PIDE, which is designed for an IDE where small delay increases are unnoticeable for the user, more performant for the future gym environment where latency induced by the interaction cycle is a bottleneck.

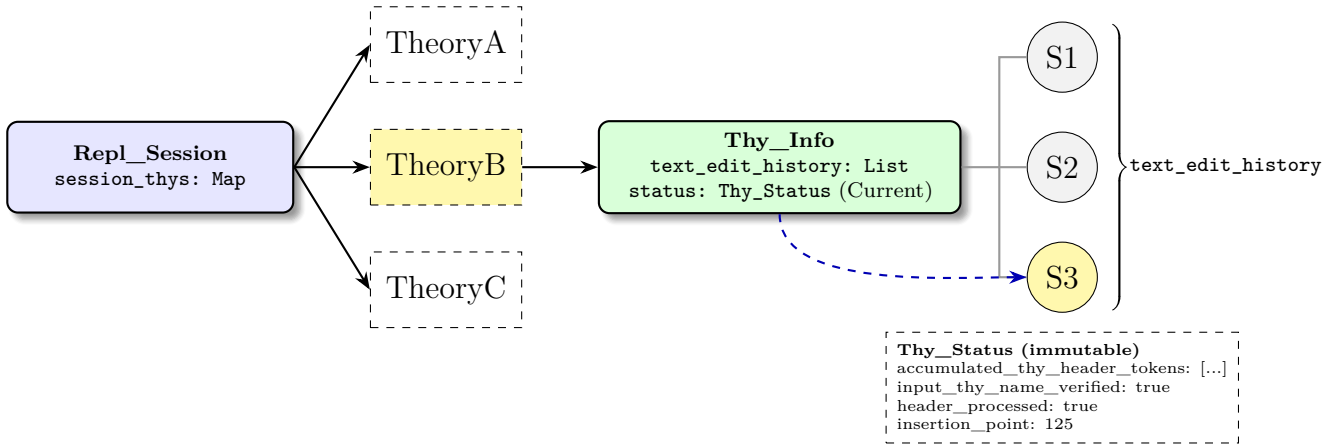


Figure 3.3: Overview of REPL management of theory information. A mapping is maintained between theory names and their corresponding `Thy_Info` objects. The `Thy_Info` object maintains the history of edits and statuses for that theory and points to the current status.

Theory headers and preamble support The theory header (`theory A imports B1 B2 ... begin`) is the only edit that doesn't neatly fit into this insertion edit architecture. While declarations, definitions and proofs must be within theories (i.e. after the header), Isar supports structured annotations (e.g. `section` and `text`) that can occur before the header. Furthermore, the token by token processing allowed by Isabelle allows the possibility for the theory header to be over multiple REPL inputs. Thus, we do not know at which REPL input the theory header will occur and over how many inputs it will occur.

These issues are mitigated by lexing the inputs and storing the tokens¹ until receive a complete theory header. Upon accumulating this header, the additional dependency edits for the imports are sent, connecting existing theories to this new one in the DAG document structure. Our helper theory containing utility Isabelle/ML functions is also imported at this stage without the user having to explicitly specify it. Subsequently, lexing is no longer required and the insertion edits are simply passed to Isabelle. This process is repeated for each new theory.

Multiple theory support The REPL supports editing multiple theories in the same session. Separate nodes are created for each theory and information such as theory header processing status, insertion point and edit history are maintained separately, with theory navigation facilitated by `enter`.

Rollback This theory information management allows rollback to be implemented simply via reverting the status to the previous version in the history. The history is a list of immutable theory status snapshots indicating details such as whether the theory header has been provided yet and whether the name in the theory header has been verified to match the entered theory name. The text inserted by the last edit is also removed via another `Document.Edit_Text`. At this point, only one edit at a time can be removed; more complex rollback mechanisms are implemented in the state saving for the gym environment (Section 3.2.2).

¹Only tokens after the keyword `theory` are stored to minimise space usage.

3.1.3 Inter-process communication (IPC)

Given that the gym environment will have a Python API, the REPL front-end is implemented in Python in preparation to build out the necessary interfaces for interaction with Isabelle. Data transfer between the two is kept to a minimum, with the Python REPL only sending Isar snippets or command invocations (e.g. rollback) to the Scala back-end. Execution of the commands and editing theories is handled by Scala, with results and errors accumulated during processing and sent back together to Python. Since both processes are run locally with mainly strings being passed between the two, the simple bridge `py4j` [22], supporting dynamic access of JVM objects from Python, was used for the IPC. Alternative IPC mechanisms such as `gRPC` [6] were considered but opted against due to the limited benefits for a local implementation and increased complexity compared to `py4j` which just required a gateway connection to be established.

3.2 Gym environment

Requirements Recap

Simple interface The gym environment API should abstract away as much of the underlying complexity as possible, allowing agents to focus on the proof generation process.

Environment state saving A mechanism should be provided to save the current state of the environment such that it can later be restored. This allows agents to efficiently conjecture and backtrack.

Proof state flexibility The full proof state as well as more simplified views should be accessible to the agents.

Data extraction Given an existing proof, the environment should be able to extract proof state-action pairs for use in supervised learning.

Sledgehammer integration Agents should be able to retrieve heuristic information from Sledgehammer to aid their premise selection.

3.2.1 API design

Gym environment function	Purpose
<code>def enter_thy(thy_name: str) -> None</code>	REPL: enter
<code>def step(isar_snippet: str) -> Result</code>	REPL: isar_snippet
<code>def rollback() -> None</code>	REPL: rollback
<code>def save_state() -> EnvStateID</code>	Saves the current state of all theories in the environment, returning a unique identifier for this state.
<code>def restore_state(state_id: EnvStateID) -> None</code>	Restores the environment state corresponding to the given identifier.
<code>def proof_finished() -> bool</code>	Checks whether the current proof has been completed (i.e. there are no open subgoals).
<code>def open_subgoals() -> list[str]</code>	Returns the minimal, user-facing proof state (i.e. the list of open subgoals) in the current theory.
<code>def proof_state(variables: bool = True, constants: bool = True, ...) -> ...</code>	Retrieves the full proof context, allowing some portions such as constants to be omitted if not needed.
<code>def sledgehammer() -> Optional[list[str]]</code>	Retrieves the steps in the first proof found by Sledgehammer (or <code>None</code> if no proof is found).
<code>def source() -> str</code>	REPL: source

Table 3.2: Shows the functions within the API of IsabelleGym. `Result` stores both outputs and errors from the Scala back-end after adding the requested Isar snippet to the current theory.

Any functions performing the same task as REPL command *A* is marked REPL: *A*.

3.2.2 Environment state saving and restoring

The state of the environment is perfectly represented by the source codes of all of the theories in the current session. Since the main application of the state saving is to allow quick conjecturing and backtracking, snapshotting the source code of all theories is an unnecessarily inefficient approach. This approach would mean copies of the full source code for all theories would have to be stored, quickly becoming infeasible with long theories and many save operations. Instead, a more efficient approach can be obtained by leveraging the insertion-only architecture.

For this discussion, the process is described from the perspective of a single theory as the state saving and restoring process and is independent for all theories. s_{current} and s_{saved} represent the states before and after restoring the saved state, respectively. If there is a theory only in one of these states, the theory is restored to its state in s_{saved} (empty but existent if the theory was not in s_{saved}). After a previous state is restored, further insertions can be made. Hence, this paradigm allows the history to diverge into a tree structure.

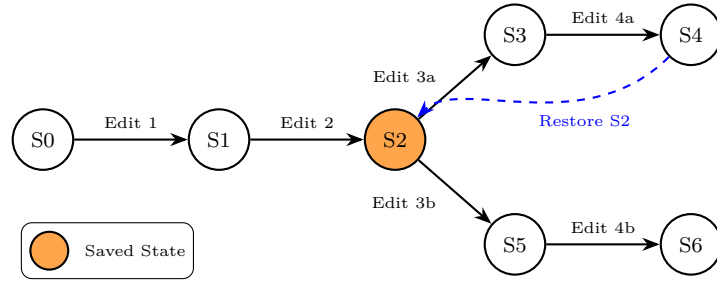


Figure 3.4: Example history tree structure. After S2 is restored, further edits are made creating another branch. If S4 is saved, its branch cannot be pruned (see below).

Algorithm 1 Restoring a saved state.

```

1 function RESTORE( $s_{\text{current}}$ ,  $s_{\text{saved}}$ )
2    $s_{\text{ancestor}} \leftarrow$  nearest common ancestor of  $s_{\text{current}}$  and  $s_{\text{saved}}$ 
3    $d_{\text{ancestor} \rightarrow \text{current}} \leftarrow$  accumulated insertions from  $s_{\text{ancestor}}$  to  $s_{\text{current}}$ 
4    $d_{\text{ancestor} \rightarrow \text{saved}} \leftarrow$  accumulated insertions from  $s_{\text{ancestor}}$  to  $s_{\text{saved}}$ 
5    $d_{\text{current} \rightarrow \text{saved}} \leftarrow \text{diff}(d_{\text{ancestor} \rightarrow \text{current}}, d_{\text{ancestor} \rightarrow \text{saved}})$ 
6   apply  $d_{\text{current} \rightarrow \text{saved}}$  edit to current theory node
7 end function

```

Nearest common ancestor The nearest common ancestor (NCA) represents the divergence point of the two histories. The NCA is found in linear time (with respect to the maximum depth of the two states in the history) by simultaneously walking back the tree from the two nodes, adding the encountered nodes to a set until a node (i.e. the NCA) is found in both sets.

Pruning Maintaining this large tree history with no removals becomes inefficient under the expected paradigm of repeated conjecturing and backtracking. If the full history is persisted and editing occurs in this style, the tree will contain many branches that are no longer accessible. Consider this example:



Figure 3.5: The history tree after interaction by a simple agent that brute forces the last step of a proof by trying a list of tactics.

The nodes in the history corresponding to trying `auto` and `simp` are no longer reachable after leaving this block. Thus, we actively prune unreachable entries in the history. On the path back from s_{current} to the NCA, all nodes until the first saved node encountered are unreachable and

so can be popped from the history. This simply extends to rollback where the last insertion edit is popped as long as it is not saved (if it is saved, only the pointer to the last edit is updated).

3.2.3 Proof state extraction

Recall from Section 2.2.1.1 that the proof state consists of the list of open subgoals (user-facing proof state) and the proof context. Often the user-facing proof state is sufficient for agents to make decisions as conditioning on the source of the theory thus far reveals the contents of the proof context. However, when large theories are imported, keeping track of the expected contents of the proof state becomes unmanageable. Hence, IsabelleGym provides a mechanism to extract the full proof state as well as just the user-facing proof state.

Difference updates Internally, the cost of repeated passing of a large proof context is somewhat mitigated by caching the last received proof context for a given theory. When the full proof state is next requested, the backend only sends a representation of what has been added and removed from the context.

Mining existing proofs Now that the full proof state is accessible via the gym environment, extracting proof state-action pairs from existing proofs becomes a simple task. Each line of the proof script is fed through the environment and before each step the proof state is extracted. A utility is provided as part of the gym especially for this purpose. This dedicated utility improves on the performance of this approach by embedding the proof state extraction commands within the proof script in advance. Then, the whole theory can be fed to the gym environment in one go, allowing the data extraction commands to be performed in parallel if there are independent subgoals. The extraction of this data is useful as it means training no longer needs any interaction with Isabelle. A sample dataset is provided alongside IsabelleGym, containing proof state-action pairs extracted from a subset of AFP proofs.

3.2.4 Sledgehammer integration

In jEdit, a dedicated panel is provided for invoking Sledgehammer within a proof context; proofs found by the various ATPs (run in parallel) are displayed in the output and can be directly inserted with a single click. This was already somewhat possible by simply running `sledgehammer` in the REPL and inspecting the output. However, for agent convenience, a dedicated API command is provided in the gym to return the results in a structured format, enabling agent checking and manipulation before insertion. IsabelleGym also supports extracting just the heuristic relevance information (see Section 2.1.4) provided by Sledgehammer to inform premise selection – one of the most challenging tasks in the proof search process.

3.3 Training an RL agent to prove theorems within Isabelle/HOL

Requirements Recap

Problem formalisation A specific problem should be selected and formalised in Isabelle/HOL such solving this problem is equivalent to completing a proof.

RL implementation After selecting a suitable RL algorithm, an agent must be trained using IsabelleGym to solve the formalised problem.

3.3.1 Problem formalisation – N -puzzle

The problem selected to demonstrate the capabilities of IsabelleGym is the N -puzzle (also known as the sliding tiles puzzle).

Problem description Given an $\sqrt{N+1} \times \sqrt{N+1}$ square grid containing tiles numbered from 1 to N (inclusive) and one empty space, find a sequence of moves that will arrange the tiles such that they are in ascending order when read in row-major order and the empty space is in the bottom right corner.

Movement rules A tile can only move into an empty space. This constraint means at most four tiles (one for each adjacent direction) are able to move in any grid configuration. Since each tile can move in at most one direction, specifying the number of the tile is sufficient to specify a move.

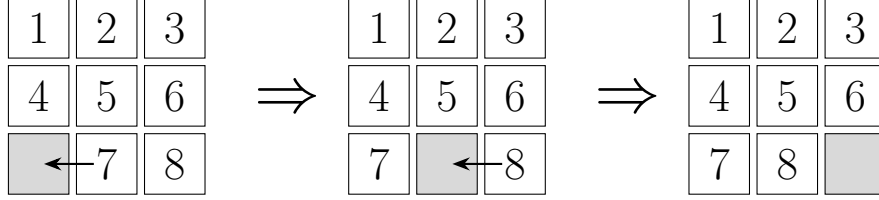


Figure 3.6: Example of an 8-puzzle. Moving 7 and then 8 solves the puzzle.

Formalisation This problem was formalised in Isabelle/HOL (see Appendix B for the full formalisation). The grid was represented by a list of tiles and concepts such as adjacency and the results of sliding moves were encoded within this representation. A reflexive, symmetric, transitive `solves` relation was the core formal entity, indicating whether there exists a sequence of moves that can transform one configuration into another. The full formalisation of the N -puzzle is provided in Appendix B.

Having successfully formalised the N -puzzle, solutions can be reframed as proofs as follows:

```

theory SlidingPuzzleSolution
  imports SlidingPuzzle
begin

definition scrambled_board :: board where
  "scrambled_board = [
    Tile 1, Tile 2, Tile 3,
    Tile 4, Tile 5, Tile 6,
    Gap,   Tile 7, Tile 8]"

theorem scrambled_solvable: "solves_n_by_n_puzzle 3
  ⇐ scrambled_board"
  unfolding scrambled_board_def
    solves_n_by_n_puzzle_def
  (* 1. solves 3 [Tile 1, Tile 2, Tile 3, Tile 4, Tile 5,
    Tile 6, Gap, Tile 7, Tile 8] (goal_board 3) *)
  apply (move 7)
  (* 1. solves 3 [Tile 1, Tile 2, Tile 3, Tile 4, Tile 5,
    Tile 6, Tile 7, Gap, Tile 8] (goal_board 3) *)
  apply (move 8)
  (* 1. solves 3 [Tile 1, Tile 2, Tile 3, Tile 4, Tile 5,
    Tile 6, Tile 7, Tile 8, Gap] (goal_board 3) *)
  apply (check_equal)
done
end

```

Figure 3.7: The scrambled board is the same as in Figure 3.6. The transformations to the subgoals (i.e. the new states of the grid) due to the movements tactics are shown by the comments.

3.3.2 Combining AlphaZero-style Monte Carlo Tree Search with temporal difference learning

Consider the N -puzzle as traversing a tree where nodes represent board configurations and edges represent moves. The root node is the initial configuration and the leaves are solved configurations. Solving this puzzle then reduces to finding a path from the root to a leaf node. The distance of a node in this context refers to minimal number of moves required to reach the goal state.² Due to the large branching factor (between 2 and 4), brute force search is infeasible. Admissible heuristics exist for this problem (e.g. sum of Manhattan distances of each tile from its goal position); hence, optimal solutions can be found using A* search. However, at distant nodes, the heuristic distances tend to be very similar regardless of which move is made, leading to a wide search wherein A* inefficiently explores the search space via breadth-first search.

Monte Carlo Tree Search (MCTS) provides an alternative approach, utilising heuristics and simulations to more efficiently explore the search space. It effectively prunes the tree focusing effort on promising paths while still maintaining a balance between exploration and exploitation. Systems like AlphaZero have combined MCTS with value and policy networks that train

²In practice, not all grid configurations for the N -puzzle are solvable [14]. Solvability is ensured here by starting from the goal state and performing a random scramble. Since the moves in this puzzle are reversible, all states reachable from a solvable state are also solvable and so every node in the search tree has a valid distance.

via RL to improve the efficiency of the search process. However, AlphaZero requires significant computational resources and specialised hardware (TPUs) to train. Instead, here a similar approach to Scheiermann *et al.* [25] is followed, training an RL agent with temporal difference learning and then wrapping this trained agent with the MCTS aspect of AlphaZero at inference time. This significantly reduces the computational requirements for training while still leveraging the power of MCTS during testing to improve performance.

3.3.2.1 Temporal difference (TD) learning

At training time, TD learning was used, with the results of self-guided playouts used to update the value network.

Value network The value network, V , is a neural network that takes an input board configuration and estimates the distance to the goal state. A grounded measure of the value of a state is given by:

$$V'(s) = \begin{cases} 0 & \text{if } s \text{ is the goal state} \\ V(s) & \text{otherwise} \end{cases}$$

ϵ -greedy playouts The agent performs playouts selecting moves using an ϵ -greedy approach. Let s be the current state, $A(s)$ the possible actions from s and s_α the state after taking action α from s . Then, the probability of selecting next action a is given by:

$$P(a = \text{random from } A(s)) = \epsilon$$

$$P\left(a = \arg \max_{\alpha \in A(s)} V'(s_\alpha)\right) = 1 - \epsilon$$

This continues until the goal state or a maximum number of moves is reached, generating a sequence of states: $s_1, s_2, \dots, s_n$. The ϵ value is annealed over training, beginning at 0.5 and decreasing to 0.01, in order to favour exploration initially and exploitation later.

Value network updates The TD error, δ , is difference between the value of the current state and expected value of the current state given the evaluation of the next state:

$$\delta_{s_i} = V'(s_i) - (1 + V'(s_{i+1})) \quad i \in \{1, \dots, n-1\}$$

Note that the evaluation from the value network is replaced with the ground truth distance if the goal state is reached. Otherwise, the process is bootstrapped using the value network evaluation of the last state. After each playout batch, the value network is updated using backpropagation and the Adam optimiser [15] with the loss based on the sum of the squares of the accumulated TD errors.

Curriculum learning The playouts only receive a ground truth reward if the goal state is found which is unlikely at the beginning of training. A naïve approach would lead to a cycle of bootstrapping on the poor value network, not finding the goal state and bootstrapping again. In order to combat the problem of sparse ground rewards, the agent is trained with increasingly difficult puzzles. The training dataset starts with states close to the goal state and then gradually further states are added.

Prohibiting immediate reversals Actions that reversed the effect of the previous action were disallowed both in training and during MCTS. This prevented redundant actions and broke simple loops that would otherwise waste time.

3.3.2.2 MCTS

At inference time, the trained value network is used to inform the MCTS algorithm. The variant of MCTS used here consists of four stages: selection, expansion, evaluation (replacing the traditional simulation) and backpropagation.

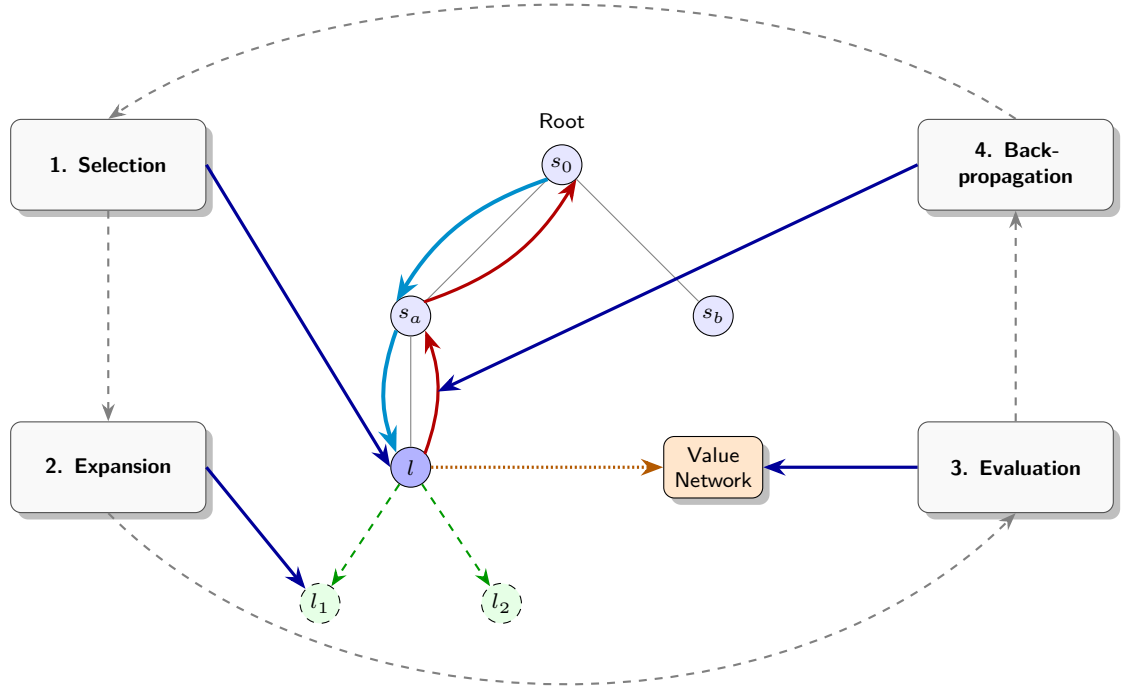


Figure 3.8: Alpha-Zero inspired Monte Carlo Tree Search utilising the value network trained via TD learning.

Search tree Each node corresponds to a state in the game that can be reached from the root (i.e. start state). Each node stores:

- Number of times the node has been visited, $N(s)$.
- Accumulated reward for the node, $W(s)$.
- The value network evaluation of node, $V(s)$.

Upon initialisation, a tree containing exactly one node for the root is created.

Selection This phase repeatedly selects child nodes beginning from the root until reaching a leaf node (a node yet to be expanded), l . The child selection policy is the same as the variant of PUCB (combining a predictor with the upper confidence bound measure) constructed by Silver *et al.* [27] for AlphaZero. Given current state s , the action corresponding to the next child to

select is given by:

$$a_{\text{next}} = \arg \max_{a \in A(s)} \left(\frac{W(s_a)}{N(s_a)} + U(s, a) \right)$$

$$U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\epsilon + \sum_{b \in A(s)} N(s_b)}}{1 + N(s_a)}$$

$$P(s, a) = \text{softmax}_{b \in A(s)}(V(s_a)) = \frac{\exp(V(s_a))}{\sum_{b \in A(s)} \exp(V(s_b))}$$

c_{puct} was a hyperparameter controlling the exploration-exploitation trade-off. It was set to 1 as in the original AlphaZero paper. The prior probability values, $P(s, a)$, is also cached in the s node to avoid repeated recalculations.

Expansion l is expanded by adding all its child nodes (for each possible $a \in A(l)$) to the tree. A further optimisation used here is that if any of the children are the goal state, the action from the root corresponding to the path l is on is immediately returned.

Evaluation Traditional MCTS uses a rollout phase wherein a random simulation is performed to get an approximation of the value of the leaf node. Here, the evaluation of l according to the value network is used directly.

Backpropagation For each state s from l back to the root, the visit counts and rewards are updated.

$$N(s) \leftarrow N(s) + 1$$

$$W(s) \leftarrow W(s) + V(l) + \text{distance}(s, l)$$

These four phases are repeated up to a maximum number of iterations and the action corresponding to the child node with the highest visit count is returned. If the number of iterations left is such that the visit counts can no longer change to the extent that the action returned changes, the search is terminated early. This optimisation can save up to half of the search time in some cases.

3.3.3 Agent training and evaluation

During training, the performance of the agent was consistently monitored by evaluating performance on a separate evaluation benchmark of unseen 3×3 puzzles, with performance measured based on the rate of solving these hidden puzzles.

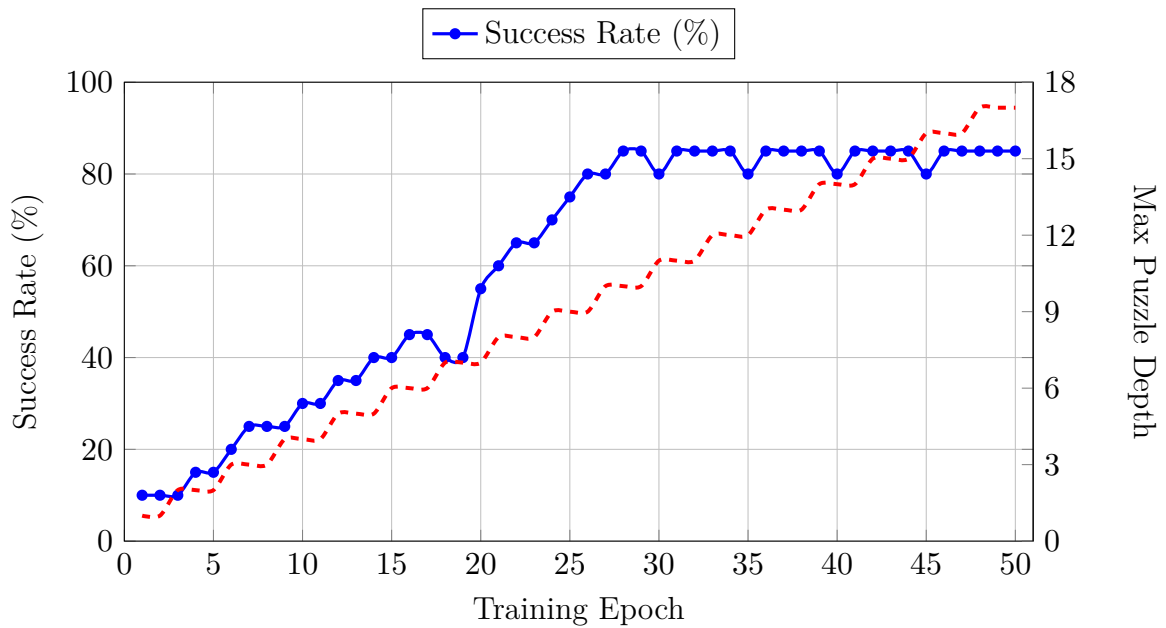


Figure 3.9: Agent performance during training. The red dashed line shows the maximum puzzle depth (difficulty) used in curriculum learning.

As Figure 3.9 shows, the agent was able to train within IsabelleGym and achieve impressive performance.

3.4 Vectorised environments

Requirements Recap

Multiple action support The vectorised environment should manage multiple independent Isabelle/HOL environments, allowing multiple actions to be performed in parallel.

Flexibility The paradigm for the vectorised environment should be flexible enough to easily switch between a scalar and vectorised environment and vice versa as required.

Efficiency Another approach to providing multiple environments is simply instantiating multiple IsabelleGym environments. This is well-suited to tasks where completely different execution paths are taken in the different environments. However, for tasks where similar execution paths are taken initially but occasional divergences occur, it duplicates a lot of work. Hence, the implementation of vectorising the current environment must be more efficient than just instantiating multiple environments.

3.4.1 Gym API changes

This extension seeks to add two modes for IsabelleGym: scalar for managing a single environment and vector for multiple environments. Extending the IsabelleGym API to support

vectorised environments requires adding two new primitives to support creating and collapsing the vectorised environment.

Gym environment function	Purpose
<pre>def vectorise(vector_size: int) -> None def scalarise(env_to_keep: int) -> None</pre>	Given the gym is in scalar mode, shift to vector mode with <code>vector_size</code> copies of the previous single environment. Given the gym is in vector mode, shift to scalar mode, removing all of the environments except <code>env_to_keep</code>.

Table 3.3: Additional IsabelleGym API functions for vectorised environments.

Once the gym is in vector mode, the `step` function takes as input a list of actions which are then performed in parallel. **Conditional execution** is supported by allowing some entries in the action list to be `None`, instructing the `step` function to take no action on the environments corresponding to those entry positions. Every other function (e.g. `enter_thy`, `is_proof_finished`) that previously only operated on a single environment now operates on all environments. Any functions in the scalar API that returned a single result now return a list of results corresponding to each environment.

3.4.2 Possible approaches

Duplicate sessions based on heap image As discussed in Section 3.1.2, Isabelle manages the group of theories available to the user via a session. The session heap image provides a persistent snapshot of the session state (including loaded theories and prover results) analogous to a compiled object. One approach to provide independent environments would be to store the current heap image and then load it into multiple sessions. While simple to implement, this approach is inefficient as multiple sessions would need to be built using the whole image (including the HOL image). Hence, this does not meet the requirement of being more efficient than just instantiating multiple gym environments.

Invisible theory duplication A more efficient approach is to duplicate the current theory. From the perspective of one of the multiple environments, it must have the only copy of that theory. The cost here would be in re-proving the results so far in the theory for each new environment. However, this is done in parallel and is insignificant if the theory is small. If intermediate results are needed by the multiple environments, an efficient way to operate within this paradigm is to have the results in one theory *A* and then create a new theory *B* importing *A* before vectorising from *B*. The results in *A* do not need to be re-proved and used by the copies of *B*.

3.4.3 Implementation

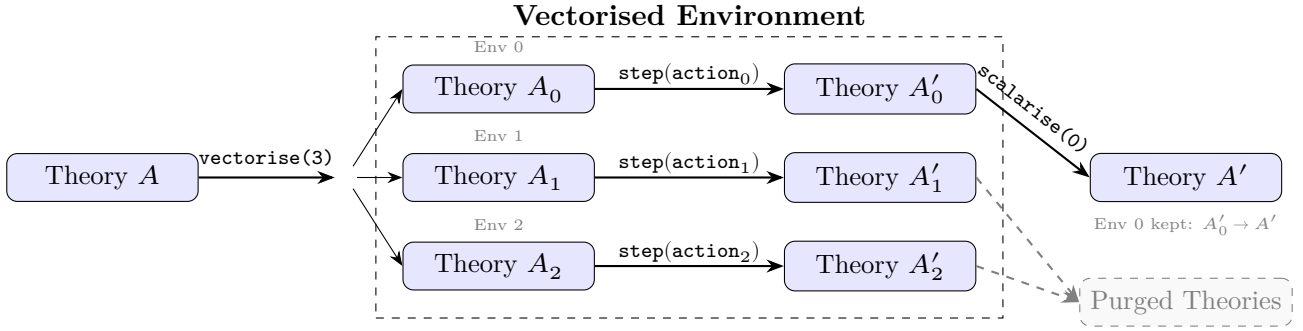


Figure 3.10: Demonstrates transitioning from a single environment to a vectorised environment, performing actions in parallel and then returning to a single environment. Each of the multiple environment operate on their own copy of the current theory. After scalarisation, only the theory manipulated by the first environment is kept and the original name is restored. Note: “Theory B ” here means the contents of theory B have been modified, but the name has not changed.

Initial vectorisation Upon the `vectorise(n)` call to the gym, the current theory is duplicated $n - 1$ times. Duplicating theory A means to create a new theory with a long random string B as its name and copy the contents of A into B .

Preserving independence Although the copy of the theory it holds may have a different name, each environment is only allowed to refer to the theory by the original name. These references are corrected automatically. A list of the actual names for the duplicate theories is maintained and explicit checks are added to prevent referencing these (although these names are not visible to the user and given the use of a long random string, it is unlikely to be referenced by accident).

Entering different theories Initially, only the environment was in when calling `vectorise` is duplicated. As all of the environments enter new theories together, any other theories entered during the vector environment lifetime are also duplicated. This on-demand duplication minimises the overhead incurred by duplication.

Rollback and saved state restoration Since state saving is done for all environments at once, the existing saving and restoration mechanism already fits in well with the theory duplication approach. A history tree is managed for each of the duplicate theories just as with the normal theories. Upon restoration of a saved state, each copy of the theory is restored to its individual state at the point of saving. The same goes for rollback.

Collapsing vectorisation All of the duplicate theories are purged from the document graph except those belonging to the environment kept by the `scalarise` call. Original theory names are restored.

3.4.4 Example usage

Recall the example from Figure 3.5 of a simple agent that tries multiple tactics sequentially to finish a proof. The added vector mode allows this process to be performed in parallel.

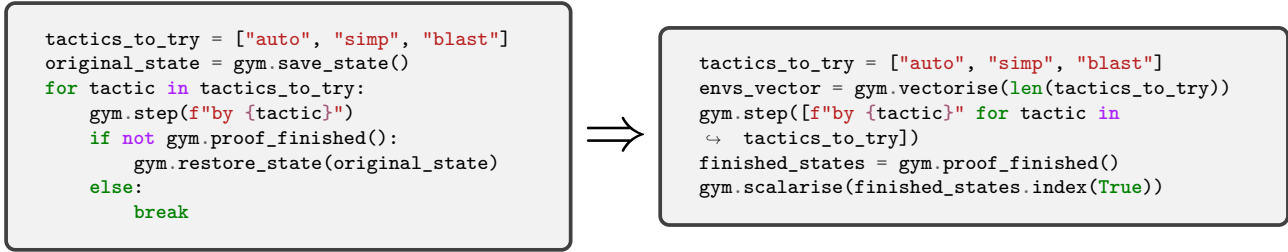


Figure 3.11: Use of a vectorised environment to try multiple tactics in parallel rather than in sequence. This example assumes at least one of the tactics will finish the proof.

3.5 Multi-agent RL support

Requirements Recap

Multiple agents IsabelleGym should support multiple agents interacting within the same theorem proving environment.

Collaborative and competitive agents Both agents that work together to create proofs and compete against each other constructing different proofs should be supported.

3.5.1 Multi-agent RL and theorem proving: collaborative vs competitive

Multi-agent theorem proving represents an extremely novel area of research. The first multi-agent system for Lean4 was MA-LoT (published March 2025) [34], which features a prover agent and a corrector agent. The prover agent uses chain-of-thought reasoning with an LLM to generate a proof script that is passed to the Lean4 environment. The feedback from Lean4 is then passed to the corrector agent which then iteratively refines the proof and analyses the feedback until a correct full proof is completed.

Agents ‘collaborate’ if their objectives are aligned, i.e. if one agent meets their objective then the other agent also does. In the context of theorem proving, this typically means that the agents are working on the same proof, helping each other to complete it (e.g. MA-LoT). This is similar to the vectorised environment where multiple agents are working within the same “theory” but the key difference here is that the agents do not work independently.

In contrast to collaborative agents, competitive agents have antagonistic objectives; the success of one agent is the failure of another. In the context of theorem proving, this generally means that the agents are working on different proofs and competing to complete their proof first. For example, there could be a prover agent and an adversarial proposer agent. The adversarial agent attempts to propose theorems that the prover agent will not be able to prove and the prover tries to produce a proof for it. This adversarial cycle continues aiming to strengthen both agents.

Both paradigms can be tackled via a **turn-taking approach**, wherein only agent is permitted to act at a given time in the current theory. This is a standard paradigm for multi-agent RL

agents allowing coordination and preventing corruption of the shared state. Alternative parallel approaches could lead to mutually exclusive actions being suggested by the multiple agents and a more complex programmer model wherein the effects of other agents would have to constantly be reasoned about alongside the one currently being implemented.

3.5.2 Agent-Environment Cycle (AEC) support

The approach followed drew inspiration from the AEC [30] model of PettingZoo, a standardised interface for multi-agent RL (similar to Gymnasium for single-agent RL). Herein, agents sequentially see the current state of the environment, take actions and the next agent is selected. Since only one agent is acting at a time, the core gym environment can be preserved and the AEC model is implemented as a wrapper around the existing IsabelleGym environment.

The basic components needed are:

- Centralised regulator that triggers agent actions, passes around shared state (supporting information sharing between the agents). The order of agent invocation in the cycle should be customisable.
- An agent interface that supports taking an action in the current environment when requested by the regulator.

Given this, the APIs that the multi-agent environment and agents implement are given below.

Multi-agent environment function	Purpose
<code>def register_agent(agent_name: str, agent: Agent) -> None</code>	Adds the given agent to the multi-agent environment. This makes it available in the AEC.
<code>def unregister_agent(agent_name: str) -> None</code>	Removes agent from those available in the AEC.
<code>def run_agent_env_cycle(agent_iter: Iterable[str], shared_state: dict) -> None</code>	Conducts the AEC, constantly selecting an agent (based on <code>agent_iter</code>) and allowing that agent to perform actions before selecting the next agent. This terminates in line with <code>agent_iter</code> .

Table 3.4: API provided by the multi-agent adaptation of IsabelleGym.

Agent function	Purpose
<code>def act(gym: IsabelleGym, additional_info: dict) -> None</code>	Allows the agent to perform its desired actions given the current multi-agent environment state. Shared state is passed via the <code>additional_info</code> dictionary.

Table 3.5: Each agent must implement a single function to take its turn when dictated by the AEC.

An example implementation of a multi-agent theorem proving system using this paradigm is shown in Appendix C.

3.5.3 Parallel agents

If the agents work on independent proofs (often the case with competitive agents), they can be run in parallel rather than each waiting for their turn. This is already possible using the existing framework so we avoid covering it in too much depth here. Briefly, parallel agents can be implemented by instantiating multiple copies of IsabelleGym. Then, each agent can be assigned its own independent environment to operate within. The agents can then run in parallel, e.g. using Python’s `multiprocessing` framework.

3.6 Repository overview

The IsabelleGym repository was built on top of the Isabelle source code. Incorporating Isabelle within the IsabelleGym repository greatly simplifies installation and allowed some minor modifications to the Isabelle/Scala API for instantiating Isabelle servers. Modularity was the key focus of the repository structure, facilitating simple composition of different components. This was particularly important given the multi-language setup employed here. Configuration files stored in the root folder, and a comprehensive `README.md` file is provided, detailing installation and usage instructions.

IsabelleGym/

└─ isabelle/.....Isabelle source code

Figure 3.12: Directory structure of the IsabelleGym repository.

4 Evaluation

In line with the project proposal, the evaluation investigates the usability, correctness and performance of IsabelleGym. Usability is evaluated by comparing IsabelleGym with existing ITP-ML environments, both for other ITPs and for Isabelle/HOL. Correctness and performance both rely on an array of tests and benchmarks.

4.1 Usability

As these gym environments are tools for researchers, usability is concerned with the following questions:

1. How easy is setting up and using the environment in its existing form?
2. What are the features supported by the environment?
3. How easy is it to extend the environment, both for adding new features and for using the latest underlying ITP features?

4.1.1 Comparison with existing learning environments for Isabelle/HOL

Here, I argue that IsabelleGym is a more usable choice for RL with Isabelle/HOL than the best pre-existing Isabelle/HOL environment, PISA, via comparison over a range of axes.

Interaction pipeline PISA utilises `scala-isabelle` in order to connect to Isabelle. As previously discussed, this is an unofficial pipeline maintained externally to Isabelle. Hence, there are no maintenance guarantees and for each new version of Isabelle, users must wait for `scala-isabelle` to be updated and then update their programs to use the new version of `scala-isabelle`, handling any changes in the API.

In contrast, IsabelleGym uses the official Isabelle/Scala and Isabelle/ML pipeline for interaction with Isabelle/HOL. Both are maintained in the core Isabelle distribution and so updates to Isabelle inherently update these layers. In `scala-isabelle`, many of the wrappers serve to emulate ML functionality in Scala. Despite Scala supporting functional programming, this emulation is very difficult and the `scala-isabelle` approach is a thick wrapper, duplicating many of Isabelle’s core structures and thereby adding overheads. IsabelleGym uses Isabelle/ML directly to manipulate low-level structures (e.g. proof states) and the first-class PIDE support for calling Isabelle/Scala functions from Isabelle/ML to transfer information. PISA is compatible with Isabelle2022 and Isabelle2021, whereas IsabelleGym is compatible with the latest releases, Isabelle2024 and Isabelle2025. In summary, this choice of using the official pipeline makes IsabelleGym much more tightly integrated with Isabelle, reduces overheads and simplifies long-term maintenance and version compatibility

Scalability For running multiple PISA instances, the recommended approach is to create

copies of the Isabelle software and the built heap images for each PISA instance in order to avoid I/O errors. This creates significant strain on storage, with each copy of AFP and Isabelle amounting to around 35 GB. In contrast, multiple IsabelleGym instances can easily be run in parallel with no duplication needed simply by instantiating the class multiple times, utilising automatic free port allocation and multiple Isabelle servers and sessions to avoid conflicts. Many of the tasks completed using a gym environment for ITP are embarrassingly parallel (e.g. proof state-action extraction from independent theories), so this scalability advantage is a significant benefit.

Advanced features IsabelleGym was built with support for a wide range of RL paradigms in mind. Features such as **vectorised environments** and **multi-agent RL** support are well-supported by IsabelleGym, whereas PISA lacks such advanced capabilities. Furthermore, the use of the official pipeline allows exposing more of the internals of Isabelle/HOL, with the ability to extract heuristic information via Sledgehammer being another feature present in IsabelleGym but not PISA.

Extensibility PISA has a complex architecture with multiple layers of Scala code, requiring significant boilerplate code for each new feature in the REPL. When adding a feature, a new string command must be defined and code must be added to dispatch it from the client. At the server, additional handler logic is needed which itself builds on `scala-isabelle`. If any Isabelle/ML code needs to be dispatched, it must be written as text in the Scala source and compiled in Scala before dispatch to Isabelle via `scala-isabelle`. Owing to its use of the official pipeline and a simple `py4j` gateway, the majority of changes when extending IsabelleGym are written directly in Isabelle/Scala or Isabelle/ML source files. On the Python side, only thin wrapper functions are then needed to expose the new Scala functionality. Hence, development load for extensions is significantly reduced with IsabelleGym.

Ease of setup IsabelleGym comes pre-packaged with the Isabelle source code and hence installation is a simple and easily automated process. PISA requires a separate installation of Isabelle, Java and the `sbt` build tool. This is a significant barrier to entry for researchers arriving from other ITPs, who may be unfamiliar with the Isabelle installation process and who just want to deal with the Python interface rather than compiling Scala code.

4.1.2 Advancements in the Isabelle-RL environment landscape during the development of IsabelleGym

Since beginning work on this project, other environments to interact with Isabelle/HOL have emerged. We briefly discuss these and show that IsabelleGym still boasts significant advantages over these environments.

DeepIsaHOL The Isabelle/RL proposal discussed in Section 1.2 has led to the development of DeepIsaHOL [9]. As of April 2025, DeepIsaHOL provides proof data extraction and a REPL interface for Isabelle/HOL. However, it is not a fully fledged gym environment and lacks the advanced features of IsabelleGym such as efficient state saving and restoration and vectorised environments. Furthermore, it too is based on `scala-isabelle` and so the previous issues with this pipeline discussed for PISA apply here.

Isa-REPL Another similar project is Isa-REPL [24], which abandons Scala altogether. Is-

abelle/ML is used directly to implement a REPL server. The direct use of Isabelle/ML allows for very fine-grained control but sacrifices the document model and PIDE features of Isabelle/Scala. Furthermore, many internal Isabelle features (e.g. parsing, state tracking) are duplicated and, unlike IsabelleGym, concurrency for the evaluation of commands within a single theory file is disabled.

4.1.3 Comparison with existing learning environments for other ITPs

The aim of this comparison is to demonstrate that IsabelleGym is of a similar standard to the state-of-the-art ML environments for other ITPs, which we surveyed in Section 2.3. These have already been used to train RL agents with impressive performance.

	IsabelleGym	Coq-Gym	Tactician	lean-gym	Lean-Dojo
Underlying ITP	Isabelle	Coq	Coq	Lean	Lean
Supported proof styles	Tactic + declarative	Tactic	Tactic	Tactic	Tactic
Structured updates	✓	✓	✓	✗	✓
State saving and restoration	✓	✓	✓	✓	✓
Proof data extraction	✓	✓	✓	✓	✓
Vectorised environments	✓	✗	✗	✗	✗
Multi-agent RL extension	✓	✗	✗	✗	✗

Table 4.1: Summary of feature support across existing ITP-ML environments and IsabelleGym.

As Table 4.1 demonstrates, IsabelleGym implements all of the features expected of a modern ITP gym environment. It even goes further in supporting advanced features such as vectorised environments and multi-agent RL. Declarative proof support is also missing from the other environments, but this is more a symptom of the limited support for this style in their underlying ITPs¹. This style is closer to human proofs and arguably easier for machine learning due to the clear exposition of logical dependencies and focus on high-level reasoning rather than fine-grained manipulations (see Section 2.1.2). Hence, supporting declarative proofs alongside tactic-based proofs is a clear advantage for IsabelleGym.

4.2 Correctness

Correctness for IsabelleGym means that the environment does not corrupt any of the underlying Isabelle functionality. This relates to accurately communicating the steps taken by agents to Isabelle, correctly passing back extracted data (outputs, errors, proof states) to agents and ensuring that any extensions non-native to Isabelle (e.g. vectorised environments) implement the correct features and do not corrupt the underlying Isabelle environment. Here, a comprehensive test suite is relied upon to demonstrate runtime correctness.

¹Lean does have support for declarative styles but it is rare in production proofs and Coq only had a declarative language for brief period before it was removed.

4.2.1 Unit and integration tests

Testing a project utilising multiple programming languages (Python, Scala and StandardML) is difficult. However, the structure of IsabelleGym means that only the Python code needs to be tested. Every component implemented in the Isabelle/Scala back-end and Isabelle/ML utilities directly contributes to a feature in the front-end and all of the effects of back-end functionality can be observed via the Python interface.

The `pytest` framework was used alongside the `expecttest` module (see Section 2.4.3), allowing testing to be conducted in a REPL-like manner where actual runtime outputs are compared to expectations. This operated especially well with IsabelleGym as there is a REPL front-end for the communication pipeline. Tests were parametrised where possible, e.g. to reuse the same test with the REPL and gym environment. Many of the tests were **compositional**. For example, tests of backtracking build upon tests for proof state extraction and error reporting, allowing the interactions between features are also tested.

All tests passed successfully. The test suite can be divided into:

- Unit tests, testing individual features (e.g. proof state extraction, error reporting, Sledgehammer integration).
- Integration tests, composing features and ensuring the interactions between them are properly handled by IsabelleGym.
- Longer end-to-end tests, simulating real-world agent inputs using proofs from AFP (see Section 4.2.2).

4.2.2 AFP simulations

Hand-crafting test cases for testing support for the many constructs and possible proof strategies in Isabelle/HOL is a tedious and demanding task. Hence, our hand-crafted testing of the core features of IsabelleGym was supplemented with simulations of real-world agent inputs.

Simulation strategy Proof steps were extracted from a subset of Isabelle’s journal of verified proofs, the [Archive of Formal Proofs \(AFP\)](#). These were then replayed in IsabelleGym, simulating the actions of an agent. Dependencies between these proofs was analysed to find opportunities for parallelisation where independent. Given all of these proofs are machine verified, no errors should be raised during the simulation and the final proof state should contain no remaining subgoals. Hence, it was possible to test the correctness of IsabelleGym by checking that these criteria were met.

Random testing of other IsabelleGym features Aside from communicating agent proof steps to Isabelle, the simulation was adjusted to incorporate random tests for other features of IsabelleGym. For example, state saving and restoration was tested by saving the state at random intervals and restoring it some random number of steps later. The success of this operation could then be tested by replaying the simulation from the point where the save was triggered. Each test simulation was seeded and the seed was reported if any tests failed, allowing for the failing case to be reproduced.

4.3 Performance

As IsabelleGym allows for live interaction, it will inevitably incur some overheads. However, we aim to show that these overheads are acceptable and that the user is able to trade off interactivity for performance.

4.3.1 Comparing IsabelleGym interaction with raw Isabelle

Here, we aim to measure the overheads incurred due to communicating with Isabelle via the IsabelleGym pipeline. This is done by first establishing a baseline using the raw `isabelle build` command, which runs the prover directly on the proof scripts without any user interaction. Then, this baseline is compared to the time taken to feed the same proof script through IsabelleGym.

Experiment setup A subset of AFP entries with minimal dependencies was selected. Recall that each AFP entry is a single session, i.e. a collection of theory files. For each entry, the Isabelle baseline was established by building the session using `isabelle build`² and the timings for checking each theory were recorded. The same group of theories were then run through IsabelleGym line-by-line, with total time for each theory similarly recorded. The number of lines passed in at once was also increased to observe how this affects the overhead. This process was repeated 10 times for each entry and the average times for raw Isabelle and IsabelleGym were compared.

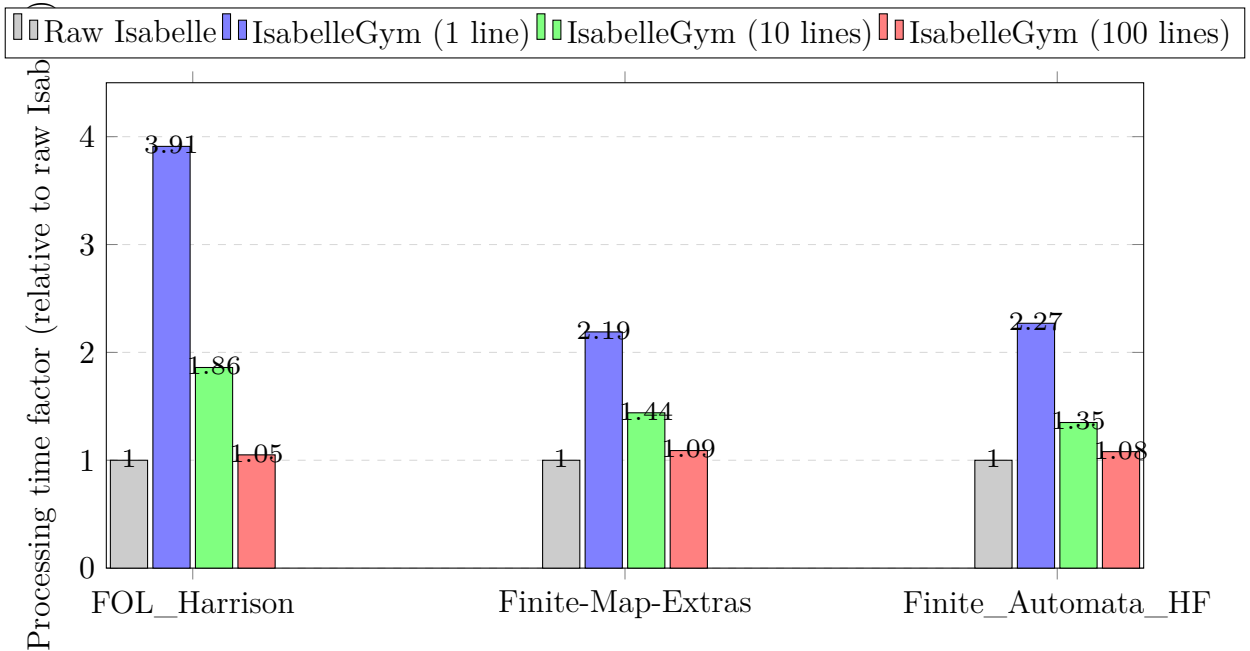


Figure 4.1: Processing time of IsabelleGym as a factor of raw Isabelle processing time across different AFP entries.

²In this experiment, raw Isabelle has the unfair advantage of having access to the whole theory at once. This allows parallelism while processing a single theory. We mitigate this by passing the additional command-line argument `-o threads=1` to disable the parallelism within theory files, emulating the step-by-step interaction with IsabelleGym. Intra-theory parallelism is also disabled for IsabelleGym to ensure it does not benefit from this parallelism as the lengths of the snippets increase.

As shown, even in the most interactive mode (1 line at a time), the overhead of IsabelleGym is only 3.91 times that of raw Isabelle. This is reasonable as agents can trade interactivity for performance by passing in multiple steps. This then bring performance very close to raw Isabelle.

4.4 Work completed

The project is analysed against the aims laid out in the Proposal.

Novel interaction pipeline A robust mode of interaction with Isabelle/HOL based on the official Isabelle/Scala API was successfully architected, dropping `scala-isabelle` and making full use of the PIDE architecture. In addition, a REPL front-end was developed, immediately exposing the capabilities of this pipeline for experimentation.

Gym environment An advanced learning environment for Isabelle/HOL was constructed, abstracting proof construction via a simple Python API with support for both declarative and procedural styles. Efficient backtracking mechanisms from simple rollback to state saving and restoration were incorporated into IsabelleGym, enabling agents to conjecture. Furthermore, the environment grants full access to the underlying proof states (including the wider proof context) and enables extracting state-action data from existing proofs. The IsabelleGym API grants Sledgehammer proof search and relevance information extraction as native actions, providing a gateway to powerful ATPs.

RL implementation The N -puzzle was formalised in Isabelle/HOL. IsabelleGym was then successfully used to train an agent to solve this puzzle using advanced RL techniques, namely TD learning paired with MCTS. This went far beyond the simple agent described in the proposal and proved the efficacy of IsabelleGym as a learning environment.

Vectorised environments (extension) An efficient vector mode was incorporated into IsabelleGym, allowing multiple actions to run in parallel and thereby enabling efficient exploration of the proof space. This is in addition to support for multiple parallel instances of the gym environment and hence IsabelleGym provides parallelism in multiple dimensions.

Multi-agent RL (extension) IsabelleGym was successfully extended to support multiple agents within collaborative or competitive theorem proving paradigms via the standard AEC [30] model.

The proposal contained many extensions and hence those judged to improve IsabelleGym the least were dropped. The MDP formalism and aspects of the [Gymnasium](#) API were incorporated into the IsabelleGym API but the exact [Gymnasium](#) API was judged to be ill-suited for an ITP-RL environment due to the infinite state and action spaces involved. The unified ITP gym was not pursued as current ITP-ML efforts are highly coupled to an underlying ITP and the gym environments for each vary significantly in their design. Hence, this extension was judged to have limited utility and risk over-abstracting the interface to the point of being unusable.

5 Conclusions

This project was a resounding success: all of the core success criteria were met and multiple extensions were implemented. The result is IsabelleGym, a novel learning environment for theorem proving agents interacting with Isabelle/HOL. IsabelleGym meets the standard of the state-of-the-art for other ITPs and significantly advances the RL landscape for Isabelle/HOL.

5.1 Lessons learnt

Given that IsabelleGym was my largest-scale and most research-intensive project to date, I have learnt much about research, project management and working with large codebases.

Minimal prototypes Planning a design in advance for components proved difficult. For the REPL, I spent too much time architecting the “perfect interface” before implementing the basic functionality. As such, several changes were needed once the challenges of implementation became clear through prototyping. In hindsight, I should have focused on producing a minimal prototype as soon as possible and then coupled the knowledge gained from developing the prototype with additional research to iteratively improve the design. This was the approach followed for the gym environment, with the knowledge gained from the prototype informing the best approach for state saving and restoration.

Usability-performance trade-off While developing IsabelleGym, several design decisions came down to the balance between usability and performance. This project has taught me that designing usable systems with reasonable performance is often better than constantly maximising raw performance with no regard for usability. For example, I initially considered using an asynchronous interface for the environment, allowing Isar snippets to be sent without waiting for the response from Isabelle. However, this would have made error handling for the agents much more difficult. In the end, I compromised for a synchronous interface which still allows delaying error messages in exchange for increased performance by allowing Isar snippets that contain many sub-commands.

Use of documentation and forums Building upon a system as mature and large as Isabelle was challenging. The many manuals and references explaining the various components of Isabelle, were of great help. However, these were often incomplete and so direct conversations with maintainers via the [Isabelle Zulip Chat](#) were invaluable. This is certainly something I wish that I had used earlier in the project as many blockers with constructing the novel interaction pipeline could have been resolved sooner.

5.2 Future work

The construction of IsabelleGym opens up several avenues for future work:

Distributed gym environment Currently, IsabelleGym is limited to a single machine. While

multicore parallelism is supported, the intense parallelism required by RL algorithms could be further supported by having multiple machines providing parallel gym environments. This would require a shift from `py4j` to an RPC system and then the gym environment will become a server, receiving commands and returning results.

Collaboration with DeepIsaHOL The author of the Isabelle/RL proposal [40], JJ Huerta y Munive of the Czech Technical University in Prague, has expressed interest in combining efforts. This collaboration has the potential to use the robust IsabelleGym pipeline and environment to train excellent tactic prediction and proof automation tooling for Isabelle/HOL.

IsabelleGym benchmark Although a small dataset of extracted proof data is provided alongside IsabelleGym, a more comprehensive benchmark from a wide variety of proofs could be created. This would greatly benefit offline learning paradigms and create a standardised metric to evaluate machine learning systems for Isabelle/HOL.

I plan to release IsabelleGym as a public Python package and to open-source the code on GitHub, allowing researches to rapidly use the environment to train RL agents and to contribute to IsabelleGym.

Bibliography

- [1] *AI achieves silver-medal standard solving International Mathematical Olympiad problems*, <https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/>, Jul. 2024.
- [2] L. Blaauwbroek, J. Urban, and H. Geuvers, “The Tactician (extended version): A Seamless, Interactive Tactic Learner and Prover for Coq,” in vol. 12236, 2020, pp. 271–277. DOI: [10.1007/978-3-030-53518-6_17](https://doi.org/10.1007/978-3-030-53518-6_17). arXiv: [2008.00120](https://arxiv.org/abs/2008.00120) [cs].
- [3] J. Blanchette, M. Desharnais, L. C. Paulson, and L. Bartl, “A User’s Guide to Sledgehammer for Isabelle/HOL,”
- [4] G. Brockman *et al.*, *OpenAI Gym*, Jun. 2016. DOI: [10.48550/arXiv.1606.01540](https://doi.org/10.48550/arXiv.1606.01540). arXiv: [1606.01540](https://arxiv.org/abs/1606.01540).
- [5] F. Gloeckle, J. Limperg, G. Synnaeve, and A. Hayat, “ABEL: Sample Efficient Online Reinforcement Learning for Neural Theorem Proving,” in *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS’24*, Oct. 2024.
- [6] *gRPC*, <https://grpc.io/>.
- [7] J. Harrison, J. Urban, and F. Wiedijk, “History of Interactive Theorem Proving,” in *Handbook of the History of Logic*, vol. 9, Elsevier, 2014, pp. 135–214, ISBN: 978-0-444-51624-4. DOI: [10.1016/B978-0-444-51624-4.50004-6](https://doi.org/10.1016/B978-0-444-51624-4.50004-6).
- [8] F. Huch, “Structure in Theorem Proving: Analyzing and Improving the Isabelle Archive of Formal Proofs,”
- [9] J. J. Huerta y Munive, *DeepIsaHOL*, <https://github.com/yonoteam/DeepIsaHOL/blob/main/src/main/scala/repl.scala>.
- [10] *Introduction to HOL4 Theorem Prover Sigma Journal of Engineering and Natural Sciences*, <https://sigma.yildiz.edu.tr/article/455>.
- [11] *Isabelle Mercurial Repository*, <https://isabelle.in.tum.de/repos/isabelle>.
- [12] A. Q. Jiang *et al.*, “Thor: Wielding hammers to integrate language models and automated theorem provers,” in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS ’22, Red Hook, NY, USA: Curran Associates Inc., Nov. 2022, pp. 8360–8373, ISBN: 978-1-71387-108-8.
- [13] A. Q. Jiang, W. Li, J. M. Han, and Y. Wu, “LISA: Language models of ISAbelle proofs,” in *6th Conference on Artificial Intelligence and Theorem Proving*, 2021, pp. 378–392.
- [14] W. W. Johnson, W. E. Story, *et al.*, “Notes on the “15” puzzle,” *American Journal of Mathematics*, vol. 2, no. 4, pp. 397–404, 1879.
- [15] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, Jan. 2017. DOI: [10.48550/arXiv.1412.6980](https://doi.org/10.48550/arXiv.1412.6980). arXiv: [1412.6980](https://arxiv.org/abs/1412.6980) [cs].
- [16] M. Mikuła *et al.*, *Magnushammer: A Transformer-Based Approach to Premise Selection*, Mar. 2024. DOI: [10.48550/arXiv.2303.04488](https://doi.org/10.48550/arXiv.2303.04488). arXiv: [2303.04488](https://arxiv.org/abs/2303.04488) [cs].
- [17] L. de Moura and S. Ullrich, “The lean 4 theorem prover and programming language,” in *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635, ISBN: 978-3-030-79875-8. DOI: [10.1007/978-3-030-79875-8_37](https://doi.org/10.1007/978-3-030-79875-8_37).

- [18] A. Newell and H. Simon, “The logic theory machine—A complex information processing system,” *IRE Transactions on Information Theory*, vol. 2, no. 3, pp. 61–79, Sep. 1956, ISSN: 2168-2712. DOI: [10.1109/TIT.1956.1056797](https://doi.org/10.1109/TIT.1956.1056797).
- [19] L. Paulson and J. Blanchette, *Three Years of Experience with Sledgehammer, a Practical Link between Automatic and Interactive Theorem Provers*. Feb. 2015. DOI: [10.29007/tnfd](https://doi.org/10.29007/tnfd).
- [20] L. C. Paulson, “Old Introduction to Isabelle,” Dec. 2021.
- [21] S. Polu, J. M. Han, K. Zheng, M. Baksys, I. Babuschkin, and I. Sutskever, *Formal Mathematics Statement Curriculum Learning*, Feb. 2022. DOI: [10.48550/arXiv.2202.01344](https://doi.org/10.48550/arXiv.2202.01344). arXiv: [2202.01344](https://arxiv.org/abs/2202.01344).
- [22] *Py4J - A Bridge between Python and Java*, <https://www.py4j.org/>.
- [23] *Pytact API documentation*, <https://coq-tactician.github.io/api/pytactician-pdoc/pytact.html>.
- [24] X. Qiyuan, *Isa-REPL*, <https://github.com/JilinHu/Isa-REPL>.
- [25] J. Scheiermann and W. Konen, “AlphaZero-Inspired Game Learning: Faster Training by Using MCTS Only at Test Time,” *IEEE Transactions on Games*, vol. 15, no. 4, pp. 637–647, Dec. 2023, ISSN: 2475-1502, 2475-1510. DOI: [10.1109/TG.2022.3206733](https://doi.org/10.1109/TG.2022.3206733). arXiv: [2204.13307](https://arxiv.org/abs/2204.13307) [cs].
- [26] S. Schulz, S. Cruanes, and P. Vukmirovic, “Faster, Higher, Stronger: E 2.3: 27th International Conference on Automated Deduction, CADE 2019,” *Automated Deduction - CADE 27*, Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), P. Fontaine, Ed., pp. 495–507, 2019, ISSN: 9783030294359. DOI: [10.1007/978-3-030-29436-6_29](https://doi.org/10.1007/978-3-030-29436-6_29).
- [27] D. Silver *et al.*, *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, Dec. 2017. DOI: [10.48550/arXiv.1712.01815](https://doi.org/10.48550/arXiv.1712.01815). arXiv: [1712.01815](https://arxiv.org/abs/1712.01815) [cs].
- [28] J. Somers, *What if writing tests was a joyful experience?* <https://blog.janestreet.com/the-joy-of-expect-tests/>.
- [29] *Tactician API*, <https://coq-tactician.github.io/api/>.
- [30] J. K. Terry *et al.*, *PettingZoo: Gym for Multi-Agent Reinforcement Learning*, Oct. 2021. DOI: [10.48550/arXiv.2009.14471](https://doi.org/10.48550/arXiv.2009.14471). arXiv: [2009.14471](https://arxiv.org/abs/2009.14471) [cs].
- [31] *The design and implementation of VAMPIRE*, https://www.researchgate.net/publication/220308761_The_design_and_implementation_of_VAMPIRE.
- [32] M. Towers *et al.*, *Gymnasium: A Standard Interface for Reinforcement Learning Environments*, Jul. 2024. DOI: [10.48550/arXiv.2407.17032](https://doi.org/10.48550/arXiv.2407.17032). arXiv: [2407.17032](https://arxiv.org/abs/2407.17032).
- [33] M. Van Otterlo and M. Wiering, “Reinforcement Learning and Markov Decision Processes,” in *Reinforcement Learning*, M. Wiering and M. Van Otterlo, Eds., vol. 12, Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 3–42, ISBN: 978-3-642-27644-6 978-3-642-27645-3. DOI: [10.1007/978-3-642-27645-3_1](https://doi.org/10.1007/978-3-642-27645-3_1).
- [34] R. Wang *et al.*, *MA-LoT: Multi-Agent Lean-based Long Chain-of-Thought Reasoning enhances Formal Theorem Proving*, Mar. 2025. DOI: [10.48550/arXiv.2503.03205](https://doi.org/10.48550/arXiv.2503.03205). arXiv: [2503.03205](https://arxiv.org/abs/2503.03205) [cs].
- [35] M. Wenzel, *Interaction with Formal Mathematical Documents in Isabelle/PIDE*, May 2019. DOI: [10.48550/arXiv.1905.01735](https://doi.org/10.48550/arXiv.1905.01735). arXiv: [1905.01735](https://arxiv.org/abs/1905.01735) [cs].
- [36] M. Wenzel, “Isabelle/jEdit – A Prover IDE within the PIDE Framework,” in *Intelligent Computer Mathematics*, J. Jeuring *et al.*, Eds., Berlin, Heidelberg: Springer, 2012, pp. 468–471, ISBN: 978-3-642-31374-5. DOI: [10.1007/978-3-642-31374-5_38](https://doi.org/10.1007/978-3-642-31374-5_38).

- [37] M. Wenzel and B. Wolff, “Isabelle/PIDE as Platform for Educational Tools,” *Electronic Proceedings in Theoretical Computer Science*, vol. 79, pp. 143–153, Feb. 2012, ISSN: 2075-2180. DOI: [10.4204/EPTCS.79.9](https://doi.org/10.4204/EPTCS.79.9). arXiv: [1202.4835](https://arxiv.org/abs/1202.4835) [cs].
- [38] M. Wenzel, “Isar — A Generic Interpretative Approach to Readable Formal Proof Documents,” in *Theorem Proving in Higher Order Logics*, Y. Bertot, G. Dowek, L. Théry, A. Hirschowitz, and C. Paulin, Eds., Berlin, Heidelberg: Springer, 1999, pp. 167–183, ISBN: 978-3-540-48256-7. DOI: [10.1007/3-540-48256-3_12](https://doi.org/10.1007/3-540-48256-3_12).
- [39] M. Wu, M. Norrish, C. Walder, and A. Dezfouli, *TacticZero: Learning to Prove Theorems from Scratch with Deep Reinforcement Learning*, Jun. 2021. DOI: [10.48550/arXiv.2102.09756](https://doi.org/10.48550/arXiv.2102.09756). arXiv: [2102.09756](https://arxiv.org/abs/2102.09756).
- [40] J. J. H. y Munive, “Isabelle/RL Project Proposal: Reinforcement learning on the Isabelle proof assistant,” 2024.
- [41] K. Yang and J. Deng, *Learning to Prove Theorems via Interacting with Proof Assistants*, May 2019. DOI: [10.48550/arXiv.1905.09381](https://doi.org/10.48550/arXiv.1905.09381). arXiv: [1905.09381](https://arxiv.org/abs/1905.09381).
- [42] K. Yang *et al.*, *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, Oct. 2023. DOI: [10.48550/arXiv.2306.15626](https://doi.org/10.48550/arXiv.2306.15626). arXiv: [2306.15626](https://arxiv.org/abs/2306.15626) [cs].

A Isabelle/ML, Isabelle/Scala and scala-isabelle

The Poly/ML implementation of Standard ML (SML) constitutes the the implementation language for the Isabelle theorem prover core. This core consists of the logical kernel, inference mechanisms, built-in tactics and internal data structures representing types and theorems among other components. The strong type system of SML greatly aids in ensuring the soundness of the logical core by enforcing that only functions with the correct signature can transform and produce theorems. Isabelle/ML refers to the use of SML within Isabelle and the Isabelle/ML code can be embedded within Isar proofs. Use of Isabelle/ML exposes the theorem prover’s internal state and allows fine-grained control over data structures representing theorems, terms, types, proof contexts and theories. Significant effort has been invested in the performance of Isabelle/ML with Poly/ML being chosen for its performance advantages, especially in the realms of garbage collection and multithreading support, over other SML implementations such as Standard ML of New Jersey (SML/NJ).

Connecting an external application to Isabelle requires a communication bridge. Since PIDE is reliant on the JVM for its standard front-end, the Scala programming language, as a hybrid functional and object-oriented language that provides interoperability with Java, is almost always used for this purpose.

Isabelle/Scala constitutes the official Scala API for interacting with Isabelle. While Isabelle/ML deals with the internal “mathematics”, Isabelle/Scala handles the “physics” of connecting the core to external components. PIDE allows structured communication between Isabelle/Scala and the prover process, with Isabelle/Scala providing higher-level abstractions over the raw YXML-based PIDE protocol. Leveraging Scala’s actor library to manage concurrency, Isabelle/Scala natively supports the asynchronous, document-oriented communication model of PIDE. The Isabelle/Scala source code is maintained alongside the Isabelle/ML code and is built in to the Isabelle distribution.

[scala-isabelle](#) is the alternative interaction medium for Isabelle from Scala. As a third party library, it is maintained outside the core Isabelle distribution by Dominique Unruh. The stated objectives of this project are to allow users to manipulate low-level Isabelle objects such as terms and theorems as though they were Scala objects and to execute code snippets within the Isabelle/ML environment. It implements its own communication protocol, injecting custom SML code into a running Isabelle process. This ML code sets up an object store and a communication channel store that the Scala code can then begin communicating with using a custom binary protocol. Offering an externally accessible, documented API, [scala-isabelle](#) greatly reduces the development effort to connect to Isabelle but this comes at the cost of duplicating several of Isabelle’s core structures, adding overhead, and relying on an unofficial library with limited maintenance guarantees.

B N -puzzle formalisation in Isabelle/HOL

```
theory SlidingPuzzle
  imports Main "HOL-Eisbach.Eisbach"
begin

datatype slot = Tile nat | Gap
type_synonym board = "slot list"

fun gap_index :: "board  $\Rightarrow$  nat option" where
  "gap_index [] = None"
| "gap_index (x # xs) =
    (if x = Gap then Some 0 else map_option Suc (gap_index xs))"

fun tile_index :: "nat  $\Rightarrow$  board  $\Rightarrow$  nat option" where
  "tile_index _ [] = None"
| "tile_index n (Tile m # xs) =
    (if n = m then Some 0 else map_option Suc (tile_index n xs))"
| "tile_index n (_ # xs) =
    map_option Suc (tile_index n xs)"

fun swap :: "nat  $\Rightarrow$  nat  $\Rightarrow$  board  $\Rightarrow$  board" where
  "swap i j b =
    (if i < length b  $\wedge$  j < length b then
      let bi = b ! i; bj = b ! j in
      b[i := bj, j := bi]
    else b)"

fun index_to_coord :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\times$  nat" where
  "index_to_coord width i =
    (i div width, i mod width)"

fun is_adjacent :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat  $\Rightarrow$  bool" where
  "is_adjacent width i j =
    (let (row_i, col_i) = index_to_coord width i;
      (row_j, col_j) = index_to_coord width j
     in ((row_i = row_j  $\wedge$  abs (int col_i - int col_j) = 1)  $\vee$ 
      (col_i = col_j  $\wedge$  abs (int row_i - int row_j) = 1)))"

inductive slide :: "nat  $\Rightarrow$  board  $\Rightarrow$  nat  $\Rightarrow$  board  $\Rightarrow$  bool" where
  slide_move:
```

```

"gap_index b = Some i  $\Rightarrow$ 
tile_index n b = Some j  $\Rightarrow$ 
is_adjacent width i j  $\Rightarrow$ 
swap i j b = b'  $\Rightarrow$ 
slide width b n b'"

fun equal_board :: "board  $\Rightarrow$  board  $\Rightarrow$  bool" where
  "equal_board [] [] = True"
| "equal_board (Tile x # xs) (Tile y # ys) =
  (x = y  $\wedge$  equal_board xs ys)"
| "equal_board (Gap # xs) (Gap # ys) =
  equal_board xs ys"
| "equal_board _ _ = False"

inductive solves :: "nat  $\Rightarrow$  board  $\Rightarrow$  board  $\Rightarrow$  bool" where
  refl: "equal_board b1 b2  $\Rightarrow$  solves width b1 b2"
| step: "slide width b1 n b2  $\Rightarrow$  solves width b2 b3  $\Rightarrow$  solves width b1 b3"

definition goal_board :: "nat  $\Rightarrow$  board" where
  "goal_board width = map Tile [1.. $\text{width}^2$ ] @ [Gap]"

definition solves_n_by_n_puzzle :: "nat  $\Rightarrow$  board  $\Rightarrow$  bool" where
  "solves_n_by_n_puzzle width b = solves width b (goal_board width)"

method move for n :: nat =
  ( rule solves.step,
    rule slide.slide_move[where n = n],
    auto simp: map_option_case )

method check_equal =
  (rule solves.refl, eval)

end

```

C Demonstration of collaborative multi-agent theorem proving

```
class Agent:
    def act(self, gym: IsabelleGym, shared_state: dict):
        """
        Perform one turn in the shared environment.
        Must be overridden by subclasses.
        """
        raise NotImplementedError

class ConjectureAgent(Agent):
    def act(self, gym: IsabelleGym, shared_state: dict):
        """Tries 'auto' for each of the open subgoals."""
        sugoals = gym.open_subgoals()
        for subgoal in sugoals:
            gym.step(f"by auto")

class CorrectorAgent(Agent):
    def act(self, gym: IsabelleGym, shared_state: dict):
        """If the proof has failed, roll back to the original state and
        ↪ use Sledgehammer."""
        if gym.proof_finished():
            print("Proof completed! No corrections need to be made.")
            return

        gym.restore_state(shared_state["proof_start_state"])
        sledgehammer_proof = gym.sledgehammer()
        for step in sledgehammer_proof:
            gym.step(step)

class SimpleMultiAgentProver(MultiAgentIsabelleGym):
    def __init__(self):
        super().__init__()
        self.register_agent("ConjectureAgent", ConjectureAgent())
        self.register_agent("CorrectorAgent", CorrectorAgent())
```

```

def prove(self, theory: str, lemma: str):
    self.gym.enter_thy(theory)
    self.gym.step(f"lemma {lemma}")
    shared_state = {"proof_start_state": self.gym.save_state()}

    self.run_agent_env_cycle(("ConjectureAgent", "CorrectorAgent"),
        ↪ shared_state)
    assert self.gym.proof_finished(), "Multi-agent proof failed!"

    # End the theory and print the proof source
    self.gym.step("end")
    print(f"Proof of lemma {lemma} was completed. Theory source:")
    print(self.gym.source())

```

Proposal

Computer Science Tripos Part II Project Proposal Coversheet

Please fill in Part 1 of this form and attach it to the front of your Project Proposal.

Part 1

Name:	Milan Tom	CRSID:	mt904
College:	Churchill	Project Checkers:(Initials)	tmj/tg
Title of Project:	IsabelleGym: A Learning Environment for Reinforcement Learning With Isabelle/HOL		
Date of submission:	21/10/2024	Will Human Participants be used?	No
Project Originator:	Wenda Li		
Project Supervisor:	Wenda Li/Prof. Lawrence Paulson		
UTO Supervisor:	Prof. Lawrence Paulson		
Director of Studies:	John Fawcett		
Special Resource Sponsor:			

Part 2

Project Checkers are to sign and comment in the students comments box on Moodle.

Part 3

For Teaching Admin use only

Date Received:		Admin Signature:	
----------------	----------------------	------------------	----------------------

IsabelleGym: A Learning Environment for Reinforcement Learning With Isabelle/HOL

1 Introduction

1.1 Gym environments

Reinforcement learning (RL) is a branch of machine learning wherein agents with knowledge of the current state of an environment take actions in that environment, receiving rewards and the updated environment state subsequent to the completion of that action. Hence, RL always has a distinction between the agents and the environment for the problem domain. The presence of this distinction means that the environment can be abstracted as a Markov Decision Process [1].

A Markov Decision Process (MDP) models executing actions (i.e. decisions) sequentially in a domain where the outcomes of these actions have some known probability distribution. It can be represented as a 5-tuple consisting of:

- State space (S): The set of states
- Action space (A): Set of possible actions
- Transition function ($P_a : S \times S \rightarrow [0, 1]$): For a given action a , $P_a(s, s')$ is the probability that taking action $a \in A$ in state s at time t will lead to being in state $s' \in S$ at time $t + 1$.
- Reward function ($R : S \times A \times S$): For a starting state s , $R(s, a, s')$ is the reward received by the agent when they reach the next state s' by taking action a in state s .
- Discount factor ($\gamma \in [0, 1]$): Constant determining whether the agent should focus on short- or long-term rewards.

Given an MDP, the RL agent can specify a policy function $\pi : S \times A \rightarrow [0, 1]$ with $\pi(s, a)$ specifying the probability that the agent will take action a in state s . The behaviour of the agent is completely specified by the policy. Once a policy has been selected, the MDP collapses to a Markov Chain. The goal for the agent is to find the π that maximises the expected value of some combination (e.g. expected discounted sum) of the probabilistic rewards.

The presence of such an abstraction enables environments for many different RL problems to provide a common interface. This allows building a “gym environment” which implements this common interface for a specific RL problem. General-purpose RL algorithms designed around the common interface can then instantly be applied to this problem. Such gym environments typically [2] handle:

- Initialising the environment and providing the initial state as an observation to the agent
- Allowing the agent to select an action
- Applying the action chosen by an agent in the current state and returning the new state, reward, whether the new state is terminal and any additional information associated with the application of the last action

It is this concept that has led to projects such as [Gymnasium](#) [3] (formerly [OpenAI Gym](#) [2]), which combines gym environments for numerous problems that all implement the same interface into one package. Users can then test and benchmark their RL algorithms on a wide variety of problem domains without having to change the core RL algorithm.

1.2 Interactive theorem provers

Interactive theorem provers (ITPs) refer to programs that collaborate with users to write a formal proof [4]. Much work has been done in the domain of automated theorem proving (ATP) to replace human users of ITPs with computers. One of the techniques used for this has been using RL to train agents to interact with these ITPs [5, 6] in order to automatically produce formal proofs of mathematical theorems. Facilitating this technique requires the existence of a gym environment for the ITP being used and as such Coq has [CoqGym](#) [7] and Lean has [lean-gym](#) [8].

Another comparable ITP is [Isabelle/HOL](#) [9, 10]. While Coq and Lean support forward proofs, Isabelle supports both forward and backward proofs. Forward proofs are those written imperatively by chaining together tactic calls, where tactics refer to functions transforming proof terms. Although tactic-style proofs provide fine-grained control over the proof states (making them suitable for program verification), they are considerably less readable and unnatural for formalised mathematics due to only revealing the goal at the end. Backward proofs are written in a declarative style, specifying the target goal upfront, and so are closer to natural language. [Isabelle/HOL](#) also boasts powerful proof automation capabilities with mature tools like [Sledgehammer](#) [11] to call external automated theorem provers and a simpler logical core (classical higher order logic without the dependent types supported by Lean and Coq which are both founded on constructive logic), easing automation tasks. The ability to seamlessly integrate both proof paradigms and maintain strong automation capabilities greatly promotes Isabelle as an ITP candidate but despite its popularity and long-standing usage, no such gym for RL exists for Isabelle. Constructing such an environment for [Isabelle/HOL](#) will be the focus of this project.

2 Description

As aforementioned, this project aims to create IsabelleGym: a “gym-like environment” for RL on top of [Isabelle/HOL](#). In this context, this environment will provide an interface with functions that allow:

- Initialising an Isabelle proof environment.
- Inspecting the current state: Output the current state of an [Isabelle/HOL](#) proof.
- Taking actions in the current state: Selecting which tactics to apply and which goals to pursue, triggering the usage of [Sledgehammer](#) and other tools, etc.
- Receiving rewards and the new state subsequent to taking actions: Receiving feedback on the validity of proof steps taken by an RL agent, the new proof state and rewards.

A key component of supporting these functions will be developing a robust pipeline to interact with Isabelle externally. Currently, we have [PISA](#) [12] which relies on [scala-isabelle](#). This project seeks to improve the currently available methods by utilising the official [Isabelle/Scala](#) pipeline to build this environment. In doing so, we will avoid having to reimplement many [Isabelle/HOL](#) core functionalities and improve performance, particularly in parallel environments. A wrapper will also be provided in a language such as Python to allow users to combine this environment with already prevalent RL libraries.

3 Starting point

I have not yet completed any work related to the project. I have some prior experience with SML and Scala (both should be aided by my much greater academic and professional experience with large OCaml codebases). I am quite comfortable working with Python and have worked on large projects using it before. I have very limited experience with [Isabelle/HOL](#) and have thus allocated time in my work plan to acquire the relevant knowledge. I expect knowledge from the Part IB Logic and Proof and Artificial Intelligence courses and the Part II Types course will complement this project. In addition, the ACS module [Proof Assistants](#) will provide much of the foundational knowledge required for working with [Isabelle/HOL](#). Hence, I have begun self-learning this course and will continue to do so during the allocated time in the work plan.

Little work has been done with RL for Isabelle. At the latest Artificial Intelligence and Theorem Proving (AITP) conference (September 2024), a project was proposed to create tooling for RL with Isabelle [13] but as of now, this project has only completed the first stage of extracting training data from Isabelle. Despite the limited progress in this area, the recent proposal of this project is a strong indicator that there is genuine interest in this topic and that a tool like IsabelleGym would make a genuine contribution in the field of ITP.

4 Success Criteria

4.1 Core objectives

1. *Novel pipeline for interacting with Isabelle/HOL externally:* Drop `scala-isabelle` and architect a read-eval-print loop (REPL) environment that allows interacting with Isabelle/HOL via the official `Isabelle/Scala` pipeline.
2. *Isabelle/HOL gym environment:* Produce an RL gym that allows an RL agent to interact with Isabelle/HOL and generate tactic-style forward proofs.
3. *Example usage of gym:* Implement a basic RL/search algorithm and demonstrate that the agent can learn to automatically prove some unseen theorem with a small proof.

4.2 Extensions

1. *MDP abstraction:* Abstract the Isabelle/HOL environment as a Markov Decision Process with a Python wrapper implementing the `Gymnasium` API.
2. *Declarative proof support:* Allow RL agents interacting with the gym environment to generate declarative-style backward proofs (in addition to supporting forward proofs as in core objective 2).
3. *Vectorised environments:* Provide a combination of multiple independent environments rather than one and handle the collection of actions in parallel.
4. *Unified ITP gym:* Create a unified gym that interacts with gyms for other proof assistants (e.g. `CoqGym` and `lean-gym`) to provide a unified interface for RL in theorem proving where the user can select their desired proof assistant.
5. *Multi-agent RL:* Add support for multi-agent reinforcement learning enabling the usage of collaborative or competitive theorem proving paradigms.

5 Evaluation strategy

Evaluation will mainly be conducted on large corpora of verified formal proofs such as the `Archive of Formal Proofs (AFP)` [14]. The `AFP` is organised like a journal and consists of many proof libraries and proofs of theorems across a wide variety of domains all mechanically checked by Isabelle. As such, corpora such as the `AFP` will enable us to evaluate IsabelleGym across the following areas:

5.1 Performance

The performance of the new pipeline for interacting with Isabelle/HOL will be benchmarked against the existing `PISA`. Proofs can be sampled from the `AFP` and we can create tests to simulate communicating each step from the sampled proof with Isabelle. Comparing the time taken to execute the same proof steps will reveal the difference in efficiency of communicating with Isabelle/HOL via the official `Isabelle/Scala` pipeline as opposed to `scala-isabelle` (used by `PISA`). Parallel performance can also similarly be compared, particularly if the `vectorised environments` extension is successfully implemented.

5.2 Correctness

Correctness can be demonstrated via a test suite consisting of:

- Unit tests: Cover basic proof steps and common tactics.
- Integration tests: Sample proofs from the corpora and simulate an RL agent iteratively feeding the steps from the sampled proof to Isabelle via IsabelleGym. We can then check that inconsistent states are not reached and the theorem is successfully declared as proved once all the steps from the sampled proof have been completed.

5.3 Usability

Case studies consisting of direct comparisons can be used to evaluate the usability compared to the state-of-the-art. For the gym environment, this will be comparisons with existing gyms for other ITPs (e.g. [CoqGym](#), [lean-gym](#)), and for the REPL, the comparison will be with [scala-isabelle](#) and [PISA](#).

6 Work plan

6.1 Core implementation

6.1.1 Weeks 1-2 (Oct 21 - Nov 03)

- *Research*: Review existing literature to improve understanding of [Isabelle/HOL](#) and reinforcement learning with ITPs as well as determine what needs to be changed to drop [scala-isabelle](#).
- *Part II*: Module Natural Language Processing - Assignment 1 (10%) [[Oct 31](#)]
- **Milestone**: Send supervisor a plan of the changes that need to be made compared to [PISA](#) to use the official [Isabelle/Scala](#) pipeline.

6.1.2 Weeks 3-4 (Nov 04 - Nov 17)

- *Core objective 1*: Produce prototype REPL to interact with [Isabelle/HOL](#) via the official [Isabelle/Scala](#) pipeline.
- *Part II*: Module Natural Language Processing - Assignment 2 (60%) [[Nov 14](#)]
- **Milestone**: Deliver write-up of prototype REPL capabilities, design and example usages to supervisor.

6.1.3 Weeks 5-6 (Nov 18 - Dec 01)

- *Core objective 2*: Roughly implement each of the core functions of the gym environment to support forward proofs.
- *Part II*: Module Natural Language Processing - Assignment 3 (30%) [[Dec 5](#)]
- **Milestone**: Deliver write-up of gym environment API and implementation details at this stage to supervisor.

6.1.4 Weeks 7-8 (Dec 02 - Dec 15)

- *Core objective 2*: Finalise gym environment and add Python wrapper.
- *Core objective 2*: Incorporate API/implementation changes suggested by supervisor.
- **Milestone**: [Isabelle/HOL](#) gym environment

6.1.5 Weeks 9-10 (Dec 16 - Dec 29)

- *Slack period*: Break for Christmas
- **Milestone**: [Isabelle/HOL](#) gym environment (same as previous)

6.1.6 Weeks 11-12 (Dec 30 - Jan 12)

- *Core objective 3*: Implement basic RL algorithm and demonstrate ability to prove basic theorem.
- *Code review*: Send code to supervisor for review.
- **Milestone**: Completed core product achieving all core success criteria.

6.1.7 Weeks 13-14 (Jan 13 - Jan 26)

- *Extension 1*: Abstract gym environment as MDP and implement the **Gymnasium** API.
- *Code review*: Amend core code based on supervisor review.
- *Evaluation*: Conduct evaluation tests based on **evaluation strategy**.
- **Milestone**: Deliver write-up of finalised API, changes required to provide this abstraction and summary of initial evaluation test results to supervisor.

6.2 Progress report

6.2.1 Weeks 15-16 (Jan 27 - Feb 09)

- Create progress report.
- Work on progress report presentation.
- *Extension 2*: Add support for declarative proofs.
- **Milestone**: Progress Report [\[Feb 7\]](#)

6.2.2 Weeks 17-18 (Feb 10 - Feb 23)

- Finish and rehearse progress report presentation.
- *Slack period*: Complete extensions **1** and **2** if incomplete and otherwise work on remaining extensions.
- *Part II*: Module Deep Neural Networks - Assignment 1 (30%) [\[Feb 21\]](#)
- **Milestone**: Progress Report Presentation [\[Feb 12-19\]](#)

6.3 Dissertation writing

6.3.1 Weeks 19-20 (Feb 24 - Mar 09)

- *Dissertation*: Write Chapters 1 (Introduction) and 2 (Preparation) and send to supervisor, co-supervisor and DoS for review.
- *Extensions*: Work on remaining if time permits.
- *Part II*: Module Deep Neural Networks - Assignment 2 (70%) [\[Mar 21\]](#)
- **Milestone**: Partial dissertation (Chapters 1 - 2)

6.3.2 Weeks 21-22 (Mar 10 - Mar 23)

- *Dissertation*: Write Chapter 3 (Implementation) and send to supervisor, co-supervisor and DoS for review.
- *Dissertation*: Amend Chapters 1-2 based on review.
- *Extensions*: Work on remaining if time permits.
- *Part II*: Module Deep Neural Networks - Assignment 2 (70%) [\[Mar 21\]](#)
- **Milestone**: Partial dissertation (Chapters 1 - 3)

6.3.3 Weeks 23-24 (Mar 24 - Apr 06)

- *Dissertation*: Write Chapter 4 (Evaluation) and send to supervisor, co-supervisor and DoS for review.
- *Dissertation*: Amend Chapter 3 based on review.
- *Extensions*: Work on any incomplete extensions if time permits.
- *Part II*: Revise for exams.
- **Milestone**: Partial dissertation (Chapters 1 - 4)

6.3.4 Weeks 25-26 (Apr 07 - Apr 20)

- *Dissertation*: Add any final work from extensions.
- *Dissertation*: Write Chapter 5 (Conclusions) and send full dissertation draft to supervisor, co-supervisor and DoS for review.
- *Dissertation*: Amend Chapter 4 based on review.
- *Part II*: Revise for exams.
- **Milestone**: Full dissertation draft

6.4 Dissertation review

6.4.1 Weeks 27-28 (Apr 21 - May 04)

- *Dissertation*: Amend full draft based on review.
- *Dissertation*: Send corrected draft to supervisor, co-supervisor and DoS for second review.
- *Source code*: Clean up any loose parts and prepare for submission.
- *Part II*: Revise for exams.
- **Milestone**: Full dissertation corrected after first review and submission-ready source code.

6.4.2 Weeks 29-30 (May 05 - May 16)

- *Slack period*: Handle final review comments and proofread whole dissertation.
- *Part II*: Revise for exams.
- **Milestone**: Final dissertation and source code submission [May 16]

7 Resource declaration.

I will use my own laptop (Lenovo Ideapad 5 15ARE05) for this project, making regular backups of the source code, write-ups and dissertation files to a private GitHub repository using Git. I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure. If this laptop fails, I have another laptop available to be used as a contingency. In case of the corruption of my GitHub repository or GitHub going down, I will also be making regular backups to the university OneDrive service. All third-party software used in this project will be open source. The current version of **AFP** (discussed in the **evaluation strategy**) has been downloaded and backed up to protect against sudden removal or outages. Additional hardware (e.g. multi-core CPUs and GPUs) may be needed to evaluate parallel performance but my supervisor/co-supervisor will arrange this via the HPC service later if needed.

References

- [1] M. Van Otterlo and M. Wiering, “Reinforcement Learning and Markov Decision Processes,” in *Reinforcement Learning*, M. Wiering and M. Van Otterlo, Eds., vol. 12, Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 3–42, ISBN: 978-3-642-27644-6 978-3-642-27645-3. DOI: [10.1007/978-3-642-27645-3_1](#).
- [2] G. Brockman *et al.*, *OpenAI Gym*, Jun. 2016. DOI: [10.48550/arXiv.1606.01540](#). arXiv: [1606.01540](#).
- [3] M. Towers *et al.*, *Gymnasium: A Standard Interface for Reinforcement Learning Environments*, Jul. 2024. DOI: [10.48550/arXiv.2407.17032](#). arXiv: [2407.17032](#).
- [4] J. Harrison, J. Urban, and F. Wiedijk, “History of Interactive Theorem Proving,” in *Handbook of the History of Logic*, vol. 9, Elsevier, 2014, pp. 135–214, ISBN: 978-0-444-51624-4. DOI: [10.1016/B978-0-444-51624-4.50004-6](#).
- [5] C. Kaliszyk, J. Urban, H. Michalewski, and M. Olšák, *Reinforcement Learning of Theorem Proving*, May 2018. DOI: [10.48550/arXiv.1805.07563](#). arXiv: [1805.07563](#).
- [6] M. Wu, M. Norrish, C. Walder, and A. Dezfouli, *TacticZero: Learning to Prove Theorems from Scratch with Deep Reinforcement Learning*, Jun. 2021. DOI: [10.48550/arXiv.2102.09756](#). arXiv: [2102.09756](#).
- [7] K. Yang and J. Deng, *Learning to Prove Theorems via Interacting with Proof Assistants*, May 2019. DOI: [10.48550/arXiv.1905.09381](#). arXiv: [1905.09381](#).
- [8] S. Polu, J. M. Han, K. Zheng, M. Baksys, I. Babuschkin, and I. Sutskever, *Formal Mathematics Statement Curriculum Learning*, Feb. 2022. DOI: [10.48550/arXiv.2202.01344](#). arXiv: [2202.01344](#).
- [9] L. C. Paulson, “The foundation of a generic theorem prover,” University of Cambridge, Computer Laboratory, Tech. Rep. UCAM-CL-TR-130, 1988. DOI: [10.48456/tr-130](#).
- [10] T. Nipkow, M. Wenzel, L. C. Paulson, G. Goos, J. Hartmanis, and J. Van Leeuwen, Eds., *Isabelle/HOL* (Lecture Notes in Computer Science). Berlin, Heidelberg: Springer, 2002, vol. 2283, ISBN: 978-3-540-43376-7 978-3-540-45949-1. DOI: [10.1007/3-540-45949-9](#).
- [11] L. Paulson and J. Blanchette, *Three Years of Experience with Sledgehammer, a Practical Link between Automatic and Interactive Theorem Provers*. Feb. 2015. DOI: [10.29007/tnfd](#).
- [12] A. Q. Jiang, W. Li, J. M. Han, and Y. Wu, “LISA: Language models of ISAbelle proofs,” in *6th Conference on Artificial Intelligence and Theorem Proving*, 2021, pp. 378–392.
- [13] J. J. H. y Munive, “Isabelle/RL Project Proposal: Reinforcement learning on the Isabelle proof assistant,” 2024.
- [14] F. Huch, “Structure in Theorem Proving: Analyzing and Improving the Isabelle Archive of Formal Proofs,”